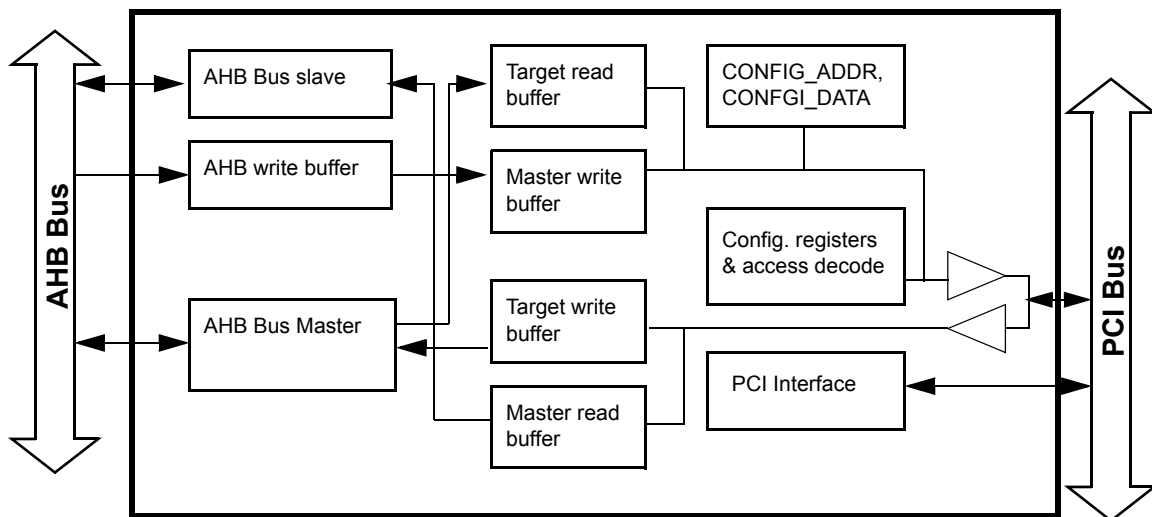




FEATURES

- Fully supports PCI specification 2.2 to 3.0 protocol.
- Supports AHB bus protocol, transfer size, burst size and response types.
- Downstream access allows AHB bus devices such as CPU to access PCI bus.
- Upstream access allows PCI bus devices to access system resources on AHB bus.
- AHB bus and PCI bus operate at independent clock domains.
- Total of six write buffers for write data posting for all interfaces.
- Supports AHB burst transfers up to 16 data words.
- Supports all AHB slave response types when functioning as AHB bus master.
- PCI interface includes bus master, bus target and configuration access initiation.
- Generates standard PCI type 0 and type 1 configuration accesses.
- Read access to PCI bus handled as delay read to prevent system deadlock.
- Automatic handling of configuration register read/write access.
- Includes all PCI specific configuration registers.
- User controlled base address register sizing and mapping.
- Retry counter to limit bus access to non-responsive target device.
- PCI status directly available to user logic for interrupt generation.
- Optional features include Address Translation, DMA, PCI bus arbiter, Cardbus, PCMCIA, MiniPCI, Power management, and Hot swap.
- Optimized for ASIC and FPGA implementations, including on-chip ARM CPU.

BLOCK DIAGRAM





DESCRIPTIONS

The EP454 PCI host bridge is designed for interfacing the ARM CPU with the PCI bus and forward data access from both the upstream and downstream directions. The host bridge consists of three functional units: AHB bus slave, AHB bus master, and PCI interface. The AHB bus and the PCI bus can operate at two different clock domains.

Any bus mastering devices on the AHB bus can also access the host bridge. The core has multiple data buffers to achieve high speed data posting, prevent bus deadlock, and allow clock domain crossing for the data.

The host bridge core allows the CPU to initialize the entire system during power-up reset using standard PCI protocol. Both type zero and type one transactions are supported. The CPU requests configuration access on the PCI bus by writing to or reading from the CONFIG_ADDR and CONFIG_DATA registers which are contained in the host bridge.

The host bridge initiates memory or IO read and write on the PCI bus upon AHB bus requests. It contains 4 write buffers, two in the AHB bus clock domain and two in the PCI clock domain, to post write data. Data can be written from the AHB bus at the same time write operation is running on the PCI bus.

Reading by the AHB bus is handled as delay read. The AHB slave retries the CPU while it is read data from the PCI bus. Instead of inserting wait state while waiting for return data, the AHB slave uses AHB bus retry to free up the AHB bus for other accesses. Once read data is read from the PCI bus, data returned to the CPU with zero wait state in subsequent read. The primary benefit of the delay read method is to prevent deadlock between the PCI bus and the AHB bus.

The host bridge functions as a PCI target when accessed by an external PCI bus master. The PCI target contains two write buffers and a read buffer to handle write posting and transfer data across the two clock domains. The request received from the PCI bus is forwarded upstream to the AHB bus through the built-in AHB bus master. When function as a AHB bus master, the host bridge supports all the bus slave response type, wait state insertion, and supports burst data transfer.



1. Signal Descriptions

The following table listed all the external I/O signals of the EP454. All signals with “#” or “_B” suffix are active low signals.

The following table lists all the signals connected to the AHB bus port of the EP454

Table 1: AHB Bus Slave Interface

Name	Type	Descriptions
HADDR[31:0]	I	AHB bus address received by the master
HBURST[2:0]	I	Burst length. Each data burst may be 4, 8 or 16 data beat long. Both cacheline wrap and linear increment are allowed.
HRDATA[31:0]	O	Read data
HREADYIN	I	Transfer ready. The bus slave uses this signal to qualify the address phase of a transfer.
HREADY-OUT	O	Transfer ready.
HRESP[1:0]	O	Bus response. Four different responses are allowed: OKAY, ERROR, RETRY and SPLIT.
HSEL[3:0]	I	AHB slave decode. Address decoding can be handled externally according to system requirement. Each bit specifies a different address space for PCI access. HSEL[0]: PCI memory access. HSEL[1]: PCI IO access. HSEL[2]: PCI CONFIG_ADDR register access. HSEL[3]: PCI CONFIG_DATA register access.
HSIZE[2:0]	I	Data bus size. It indicates the data size is 8-bit, 16-bit or 32-bit.
HTRANS[1:0]	I	Transfer type. It can be NONSEQUENTIAL, SEQUENTIAL, IDLE or BUSY.
HWDATA[31:0]	I	Write data.
HWRITE	I	Read/write indicator. It is high for write and low for read.



Table 2: AHB Bus Master Interface

Name	Type	Descriptions
M_HADDR[31:0]	O	AHB bus address driven by the master
M_HBAR[1:0]	O	Sideband signal used in forward PCI transaction to AHB bus. It indicates which BAR address matches with the current transaction.
M_HBURST[2:0]	O	Burst length. Each data burst may be 4, 8 or 16 data beat long, depending on burst internal buffer size and actual number of data to be transferred. Both cacheline wrap and linear increment are allowed on the AHB bus but the AHB-PCI bridge generates liner increment transfer only.
M_HBUSREQ	O	Bus request. Indicates the master's intention of acquiring the AHB bus to transfer data.
M_HGRANT	I	Bus grant. Driven by the system arbiter to indicate to the master that it has bus ownership.
M_HLOCK	O	Bus lock. The host bridge does not request lock access so this signal is driven low by the host bridge.
M_HRDATA[31:0]	I	Read data
M_HREADY	I	Transfer ready. The signal is driven by the slave to signal data transfer.
M_HRESP[1:0]	I	Bus response. Four different responses are allowed: OKAY, ERROR, RETRY and SPLIT.
M_HSIZE[2:0]	O	Data bus size. It indicates the data size is 8-bit, 16-bit or 32-bit.
M_HTRANS[1:0]	O	Transfer type. It can be NONSEQUENTIAL, SEQUENTIAL, IDLE or BUSY.

**Table 2: AHB Bus Master Interface**

Name	Type	Descriptions
M_HWDAT A[31:0]	O	Write data.
M_HWRITE	O	Read/write indicator. It is high for write and low for read.

Table 3: PCI Bus Interface Signals

Name	Type	Descriptions
AD[31:0]	I/O	Address and Data are multiplexed on the same PCI pins. A bus transaction consists of an address phase followed by one or more data phases. The PCI host bridge supports both read and write bursts.
C/BE[3:0]#	I/O	Bus Command and Byte Enables. They are multiplexed on the same PCI pins. During the address phase of a transaction, C/BE#[3:0] define the bus command. During the data phase, C/BE#[3:0] defines byte enables.
DEVSEL#	I/O	Device Select. When actively driven, indicates the driving device has decoded its address as the target of the current access. As an input, DEVSEL# indicates whether any device on the bus has been selected.
FRAME#	I/O	Cycle Frame. It is driven by the host bridge to indicate the beginning and duration of an access. FRAME# is asserted to indicate a bus transaction is beginning. While FRAME# is asserted, data transfers continue. When FRAME# is de-asserted, the transaction is in the final data phase.
GNT#	I	Grant. It indicates to the host bridge that access to the PCI bus has been granted.
IDSEL	I	Initialization Device Select. It is used as a chip select during configuration read and write transactions
IRDY#	I/O	Initiator Ready. It indicates the host bridge's ability to complete the current data phase of the transaction. IRDY# is used in conjunction with TRDY# to indicate a completion of one data transfer.



Table 3: PCI Bus Interface Signals

Name	Type	Descriptions
PAR	I/O	Parity. It is an even parity across AD[31:0] and C/BE[3:0]# signals. It is driven by the host bridge for address and write data phases, and is driven by the target for all read data phases.
PERR#	I/O	Parity Error. It is driven by the device on the PCI bus that receives data to report an error condition.
REQ#	O	PCI Bus Request. Driven by the host bridge to indicate to the arbiter that it needs to use the bus.
SERR#	I/O	System Error. It is driven by the device on the PCI bus that receives address parity errors, data parity errors on the special cycle command, or any other system error.
STOP#	I/O	Stop indicates the current target is requesting the host bridge to stop the current transaction due to unusual conditions. Depending on the timing of TRDY# and DEVSEL#, it may be used to signal disconnect, retry or target abort.
TRDY#	I/O	Target Ready. Indicates the target's ability to complete the current data phase of the transaction. It is used in conjunction with IRDY#. Data is transferred on the PCI bus only when both TRDY# and IRDY# are asserted at the same cycle.



Table 4: Miscellaneous Signals

Name	Type	Descriptions
BAR0_SETUP[34:16]	I	Setup for BAR0. This is a static input allows the user to setup BAR0. Bit Description 34 1=prefetchable, 0=not prefetchable. Used only when BAR is memory mapped. 33 1=enable BAR0, 0=disable 32 1=map to PCI memory, 0=map to PCI IO 31:16 BAR size, “0111111111111111” = 2Gbytes “0011111111111111” = 1Gbytes “0001111111111111” = 512Mbytes “0000111111111111” = 256Mbytes “0000011111111111” = 128Mbytes “0000001111111111” = 64Mbytes “0000000111111111” = 32Mbytes “0000000011111111” = 16Mbytes “0000000001111111” = 8Mbytes “0000000000111111” = 4Mbytes “0000000000011111” = 2Mbytes “0000000000001111” = 1Mbytes “0000000000000011” = 512Kbytes “0000000000000001” = 256Kbytes “00000000000000001” = 128Kbytes “00000000000000000” = 64Kbytes
BAR1_SETUP[33:16]	I	Setup for BAR1. This is a static input allows the user to setup BAR1. Bit Description 34 1=prefetchable, 0=not prefetchable. Used only when BAR is memory mapped. 33 1=enable BAR1, 0=disable 32 1=map to PCI memory, 0=map to PCI IO 31:16 BAR size, same definition as in BAR0_SETUP.
BIG_ENDIAN	I	Static signal to indicate that the AHB bus is running at big endian mode. If it is low, the AHB bus is running at little endian mode.



Table 4: Miscellaneous Signals

Name	Type	Descriptions
CLASS_CODE[23:0]	I	Static input specifying the class_code field of the configuration register.
DEVICE_ID [15:0]	I	Static input specifying the device_id field of the configuration register.
H_INTA#	I	PCI interrupt request. The user logic asserts this signal to triggered PCI interrupt (provided that PCI interrupt is enabled).
HS_FAST	I	Static input indicating that the AHB bus clock is faster than the PCI clock. When HS_FAST is asserted (high), the PCI core starts the master write operation on the PCI bus as soon as possible, knowledge that write data transfer from AHB clock domain is faster the transfer rate on the PCI bus.
INTERRUPT_PIN[7:0]	I	Static input specifying the interrupt_pin field of the configuration register.
PREFETCH_LIMIT[1:0]	I	Static input to control the way the PCI target prefetch read data from the AHB interface in the case of PCI burst read. 00 : Prefetch up to the next 4 word boundary. 01 : Prefetch up to the next 8 word boundary. 10 : Prefetch up to the next 16 word boundary. 11 : No prefetch, read only one word and disconnect.
REVISION_ID[7:0]	I	Static input specifying the revision_id field of the configuration register.
RUN_66MHZ	I	Static input specifying the 66Mhz field of the configuration register.
SEL_23	I	Static input select between PCI 2.2 or 2.3 version. This signal is high for version 2.3 and low for version 2.2 compliant
STATUS15, STATUS14, STATUS13, STATUS12, STATUS11, STATUS8, STATUS_RETRY	O	Output signals reflect the values of the PCI status register bit 8, and 11 - 15. STATUS_RETRY reflects the value of the retry overflow bit in the retry control register.



Table 4: Miscellaneous Signals

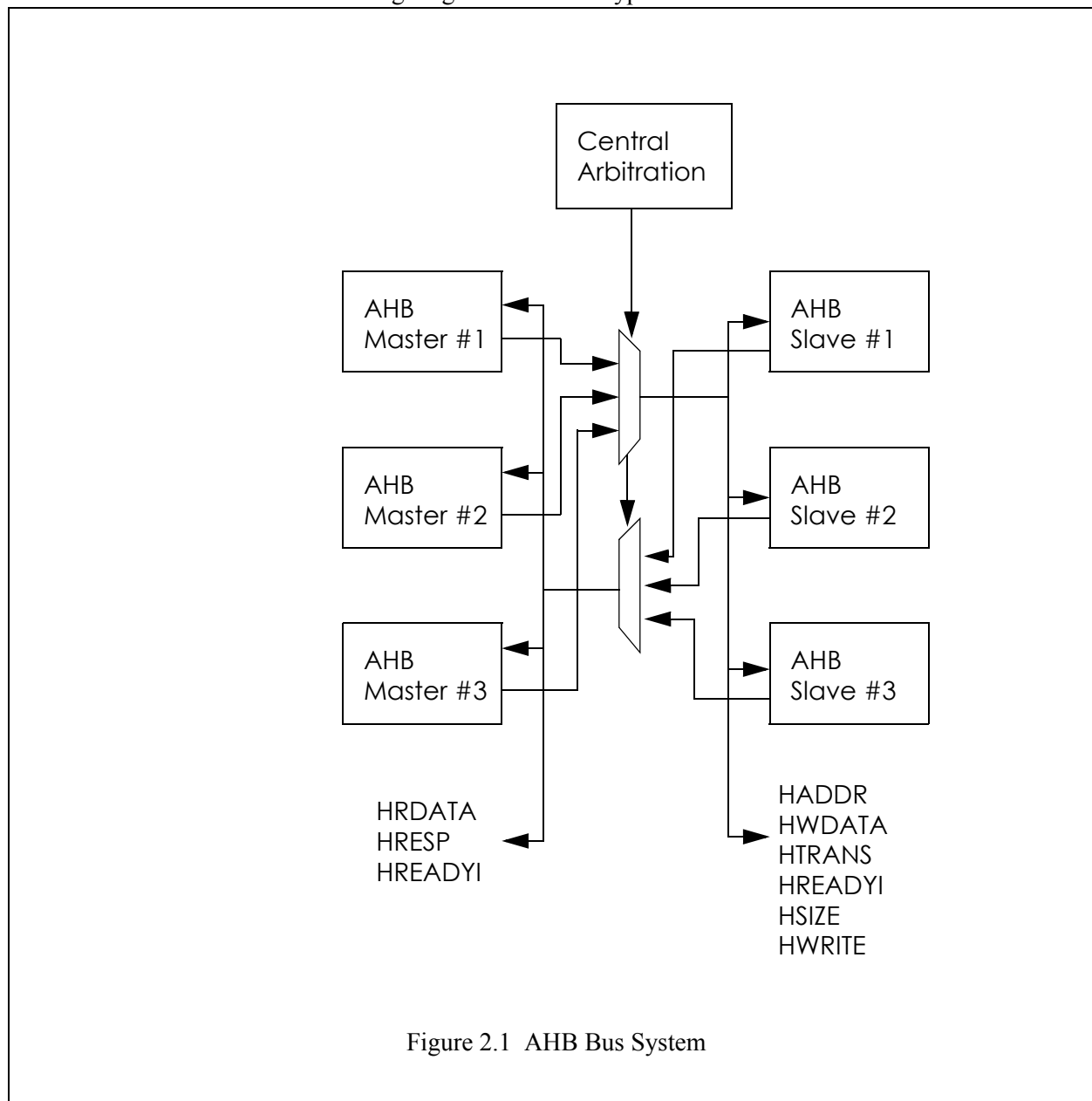
Name	Type	Descriptions
SUBSYSTEM_ID[15:0]	I	Static input specifying the subsystem_id field of the configuration register.
SUBVENDOR_ID[15:0]	I	Static input specifying the subsystem vendor_id field of the configuration register.
VENDOR_ID[15:0]	I	Static input specifying the vendor_id field of the configuration register.
PCICLK	I	PCI bus clock input. All PCI bus signals are synchronous to PCICLK. The AHB-PCI bridge allows PCICLK and SYSCLK to be asynchronous to each other.
RESET#	I	Reset input to the core
SYSCLK	I	System clock input. All AHB bus signals are synchronous to SYSCLK.



2. AHB Bus Protocol

The AHB bus is designed with a central arbitration unit to control routing of different signals between multiple masters and slaves. Each master or slave device receives dedicated signals to and from the arbitration unit with no tri-state signals.

The following diagram illustrates typical bus architecture.





2.1 Basic Transfer

An AHB transfer contains the address phase and data phase. For single transfer, only one data phase follows an address phase. For burst transfer, multiple data phase follows an address phase.

The master initiates a transfer by driving the address and control signals after the rising clock edge. It is sampled by the slave at the following rising clock edge to complete the address phase.

The address phase may overlap the data phase of a previous data transfer. However, an address phase is valid only when it occurs at the same time or after the last data phase of a previous transfer. Each slave uses the HREADYI input to qualify an address. This will be described in more detail later.

The data phase timing is controlled by the slave being access. The slave may de-assert the HREADY signal to insert wait state in conjunction with the HRESP signals to indicate different types of responses. The following diagram shows a basic transfer.

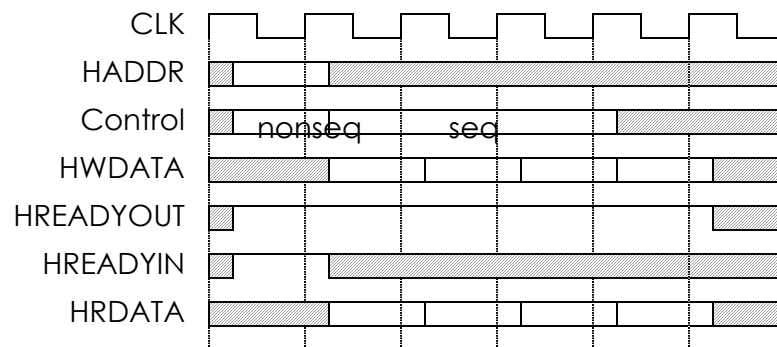


Figure 2.2 Transfer with no wait state

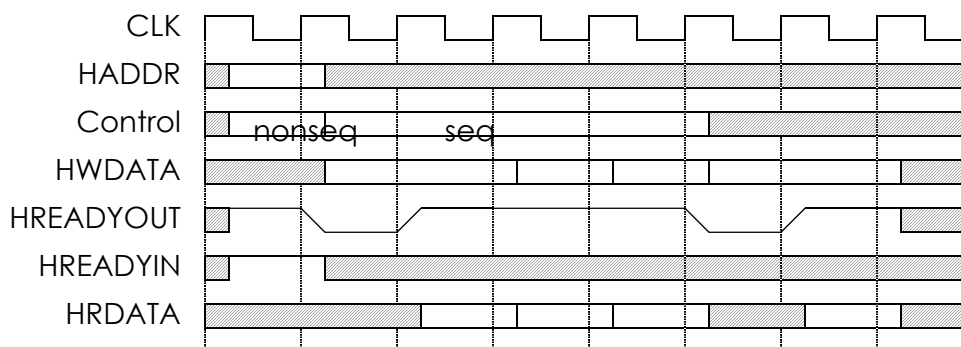


Figure 2.3 Transfer with wait state

2.2 Transfer type

The master uses the transfer type signals (HTRANS) to indicate the bus access. The HTRANS signal is driven by the current bus master and is encoded with the following combinations:

Table 5: Transfer Types

HTRANS[1:0]	Description
0 0	IDLE Indicates that no data transfer is required. The IDLE transfer type is used when a bus master is granted the bus, but does not wish to perform a data transfer. Slaves must always provide a zero wait state OKAY response to IDLE transfers and the transfer should be ignored by the slave.



Table 5: Transfer Types

HTRANS[1:0]	Description
0 1	BUSY The BUSY transfer type allows bus masters to insert wait state cycles in the middle of bursts of transfers. This transfer type indicates that the bus master is continuing with a burst of transfers, but the next transfer cannot take place immediately. When a master uses the BUSY transfer type the address and control signals must reflect the next transfer in the burst. The transfer should be ignored by the slave. Slaves must always provide a zero wait state OKAY response, in the same way that they respond to IDLE transfers.
1 0	NONSEQ Indicates the first transfer of a burst or a single transfer. The address and control signals are unrelated to the previous transfer. Single transfers on the bus are treated as bursts of one and therefore the transfer type is NONSEQUENTIAL.
1 1	SEQ The remaining transfers in a burst are SEQUENTIAL and the address is related to the previous transfer. The control information is identical to the previous transfer. The address is equal to the address of the previous transfer plus the size (in bytes). In the case of a wrapping burst the address of the transfer wraps at the address boundary equal to the size (in bytes) multiplied by the number of beats in the transfer (either 4, 8 or 16).

The following diagram shows the usage of the 4 transfer types. At cycle 1, the master is not ready to issue any commands so the transfer type of IDLE is issued. Cycle 2 is the beginning of an access so NONSEQ is issued. It is a single access and the master issue another access at cycle 3 using NONSEQ transfer type again. Since this access is a burst data access, the master issues SEQ in cycle 4 to continue the transfer. Wait state is inserted by the master at cycle 5 and 6 with the BUSY transfer type. Finally two more SEQ transfer type at cycle 7 and 8 com-



pletes the data phase of this burst access for four data.

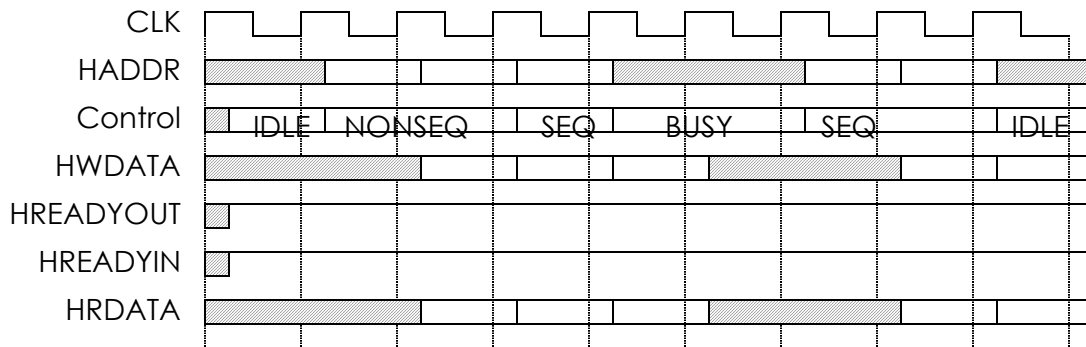


Figure 2.4 Transfer Types



2.4 Slave Response to Data Phase

Data phase of a transfer is controlled both by the master and by the slave. This section will describe the slave's control to data transfer. Master can also control the timing of data transfer by inserting the BUSY cycle. This will be described in later sections.

Once a slave receives a qualified address phase, it starts to process the data transfer. For write access, the write data is provided by the master one cycle after the qualified address phase. The write data is driven on the HWDATA bus until the slave asserts HREADY signal. The system arbiter is responsible to route the HREADY signal from this slave to the master.

If the slave cannot complete the write operation immediately, it can insert wait state by de-asserting the HREADY signal for one or more cycles until it is ready to complete the transfer. The master is required to continue to drive the valid data on the HWDATA bus during the wait state. For burst data transfer, the master will issue the SEQ transfer type after the qualified address phase. The next address is driven by the master on the HADDR bus and the slave will continue the data transfer using the HREADY signal.

Read data transfer is handled by the HREADY signal similar to write transfer. Wait state can be inserted by the slave until it drives the valid data on the HRDATA bus.

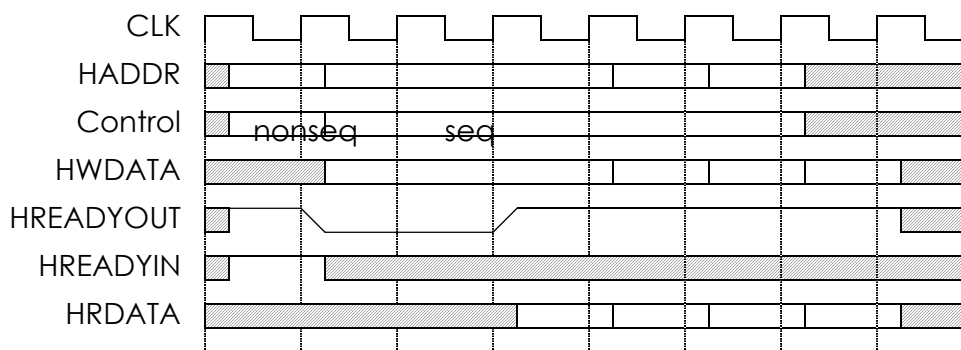


Figure 2.6 Read/write transfer with wait state



2.5 Burst Data Transfer

The AHB bus supports transfer size of four, eight and sixteen-beat and undefined-length bursts and single transfers. Both incrementing and wrapping bursts are supported in the protocol:

- Incrementing bursts access sequential locations and the address of each transfer in the burst is just a linear increment of the previous address.
- Wrapping bursts access location as if the data are within one cache line without crossing the cache line boundary. If the start address of the transfer is not aligned to the total number of bytes in the burst (size x beats) then the address of the transfers in the burst will wrap when the boundary is reached. For example, a four-beat wrapping burst of word (4-byte) accesses will wrap at 16-byte boundaries. Therefore, if the start address of the transfer is 0x34, then it consists of four transfers to addresses 0x34, 0x38, 0x3C and 0x30.

Bursts must not cross a 1kB address boundary. It is the responsibility of the master not to attempt to start a fixed-length incrementing burst which would cross this boundary.

It is acceptable to perform single transfers using an unspecified-length incrementing burst which only has a burst of length one.

The following table list the allowed burst transfer size:

Table 6: Burst Transfer Size

HBURST[2:0]	Type	Description
000	SINGLE	Single transfer
001	INCR	Incrementing burst of unspecified length
010	WRAP4	4-beat wrapping burst
011	INCR4	4-beat incrementing burst
100	WRAP8	8-beat wrapping burst
101	INCR8	8-beat incrementing burst
110	WRAP16	16-beat wrapping burst
111	INCR16	16-beat incrementing burst



2.6 Data Size

The data bus width is 32-bit but the master has the optional to transfer data with smaller data width. The HSIZE and the HBURST input together determine the data transfer size. Total data transfer equals data size times burst size.

Table 7: Data Size Encoding

HSIZE[2:0]	Description
000	Byte
001	Halfword (16-bit)
010	Word (32-bit)
Others	Reserved

The HSIZE signal supports controls the transfer size and the byte lane used for each transfer is according to the following table:

Table 8: Byte Lane Assignments

Width	Starting address	Data [31:24]	Data [23:16]	Data [15:8]	Data [7:0]
byte	0				X
byte	1			X	
byte	2		X		
byte	3	X			
halfword	0			X	X
halfword	2	X	X		
word	0	X	X	X	X



2.7 Retry and Slave Response

Previous section described the slave's responses to transfer with the HREADY signal. In addition the HREADY signal, the slave also provide the response type using the HRESP signals to further indicate its response types:

Table 9: Slave Responses

HRESP[1:0]	Type	Description
0 0	OKAY	When HREADY is high this shows the transfer has completed successfully. The OKAY response is also used for any additional cycles that are inserted, with HREADY low, prior to giving one of the three other responses.
0 1	ERROR	This response shows an error has occurred. The error condition should be signalled to the bus master so that it is aware the transfer has been unsuccessful. A two-cycle response is required for an error condition.
1 0	RETRY	The RETRY response shows the transfer has not yet completed, so the bus master should retry the transfer. The master should continue to retry the transfer until it completes. A two-cycle RETRY response is required.
1 1	SPLIT	The transfer has not yet completed successfully. The bus master must retry the transfer when it is next granted access to the bus. The slave will request access to the bus on behalf of the master when the transfer can complete. A two-cycle SPLIT response is required.

The ERROR, RETY and SPLIT response required at least two cycles to process. The slave is required to insert wait state prior to signaling any one of the three



responses. The following timing diagram shows a retry response.

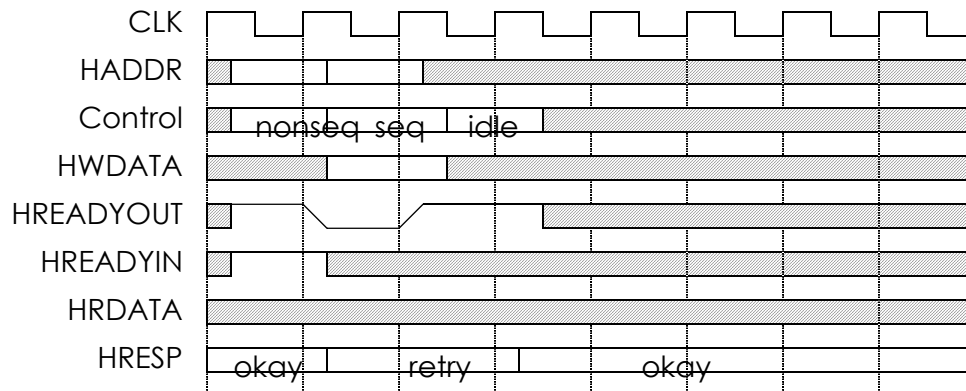


Figure 2.7 Retry



2.8 Master Wait State

The BUSY transfer type allows bus masters to insert wait states in the middle of bursts of transfers. This transfer type indicates that the bus master is continuing with a burst of transfers, but the next transfer cannot take place immediately. When a master uses the BUSY transfer type the address and control signals must reflect the next transfer in the burst. The transfer should be ignored by the slave. Slaves must always provide a zero wait state OKAY response, in the same way that they respond to IDLE transfers.

The following diagram shows the master insert wait state in the middle of a burst transfer by using the BUSY transfer type.

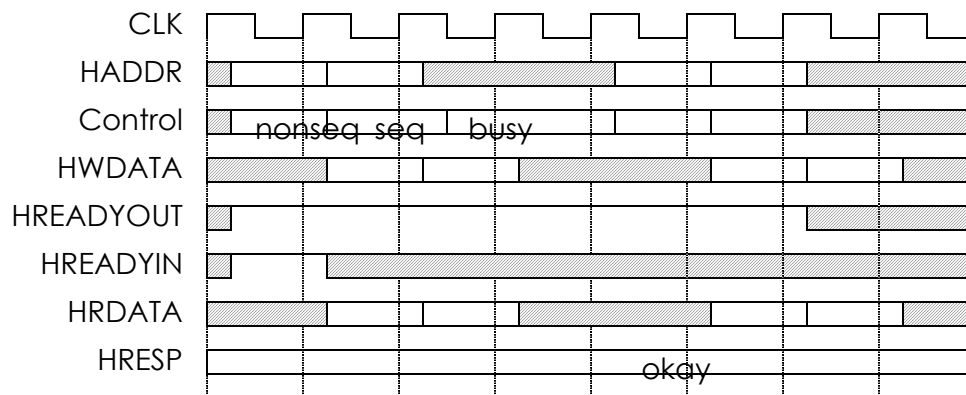


Figure 2.8 Master wait state



2.9 Early Burst Termination

The master can terminate a burst transfer before all the data transfer are completed. The slave can determine when a burst has terminated early by monitoring the HTRANS signals and ensuring that after the start of the burst every transfer is labelled as SEQ or BUSY. If a NONSEQ or IDLE transfer occurs then this indicates that a new burst has started and therefore the previous one must have been terminated.

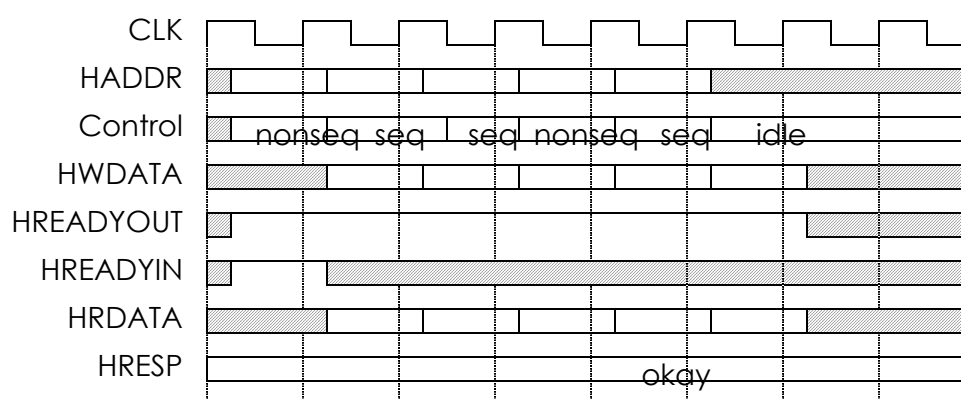


Figure 2.9 Burst terminated by NONSEQ or IDLE transfer



3. PCI Master Functional Description

3.1 Upstream and Downstream Requests

The EP454 is a bi-directional bridge between the AHB bus and PCI bus. Request from the CPU on the AHB bus can be forwarded “downstream” to the PCI bus. Request flows in this direction allows the CPU on the AHB bus to read and write to PCI devices. In the opposite “upstream” direction, request from the PCI bus by other PCI bus masters are forwarded to the AHB bus. This direction of transfer allows external PCI bus masters to read and write to AHB bus devices.

It is important to note that the terms “upstream” and “downstream” refer to the direction of the request, not the direction of data transfer. A downstream write request have both the request and the transfer data flows from the AHB bus to the PCI bus. A downstream read request have the request flows from the AHB bus to the PCI bus while the transfer data flows from the PCI bus to the AHB bus. In this document, both transfers are called downstream request.

In downstream request, the AHB-PCI host bridge functions as a slave on the AHB bus and a master on the PCI bus. In upstream request, the AHB-PCI host bridge functions as a target on the PCI bus and as a master on the PCI bus.

The following sections of the data sheet first describes downstream requests, followed by upstream requests.

3.2 Downstream Write Posting

The AHB-PCI host bridge contains 4 write buffers for posting write data. Two write buffers operates at the AHB bus clock domain. These two write buffers operates in parallel so that write data can be posted into one write buffer from the AHB bus while data is read from the other write buffer to transfer downstream. Each write buffer can store up to 64 bytes of data. These two write buffers are designed in such way that they received data of 8, 16, and 32-bit data width from the AHB bus and transfer downstream in 32-bit data width.

Data from these two write buffers are then transferred to the PCI clock domain through two more write buffers. Because the bus clock frequency of the AHB bus and PCI bus are different and the bus utilization may various, the 4 write buffers improve transfer efficiency of the host bridge.



The following diagram illustrate the write buffer arrangements.

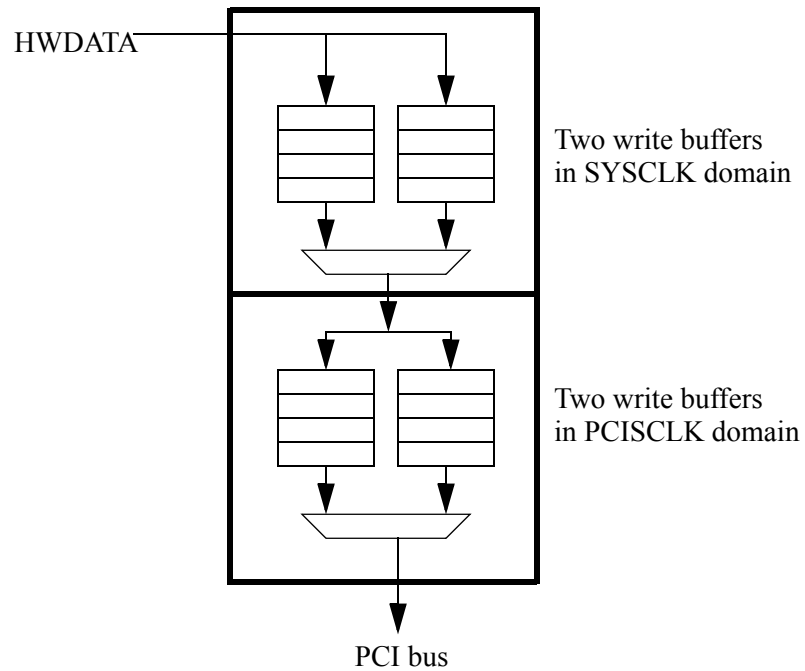


Figure 3.1 Downstream write buffers



3.3 Write Access from AHB Bus

When the AHB-PCI host bridge receives write request from the AHB bus master, it decodes the request to determine if the request should be dispatched to PCI bus.

Once the destination is determined and if write buffer is available to hold the data, the AHB-PCI host bridge posts the write data into the write buffer and asserts HREADY to complete each data word transfer. Burst length up to 16 words (64 bytes) is allowed by the AHB bus protocol.

After data is posted into the write buffer, the AHB-PCI host bridge forwards the write request and write data downstream to the next stage.

3.4 Write Access Initiated on PCI Bus

After data is posted in the write buffers, the AHB-PCI host bridge uses asynchronous request/acknowledge protocol to forward the request across the AHB and PCI bus clock domains. Both the request and acknowledge signals that travels across the clock domains are single edge transition signals driven by flip-flops. They are double synchronized by two flip-flops in series in the receiving clock domains to avoid false signaling and metastable states.

The write data are transferred to the second stage of write buffers so that they can be read from the PCI bus clock domain.

The following diagrams shows write data transfer from AHB bus to PCI bus.



The following examples shows a burst write of 4 data words.

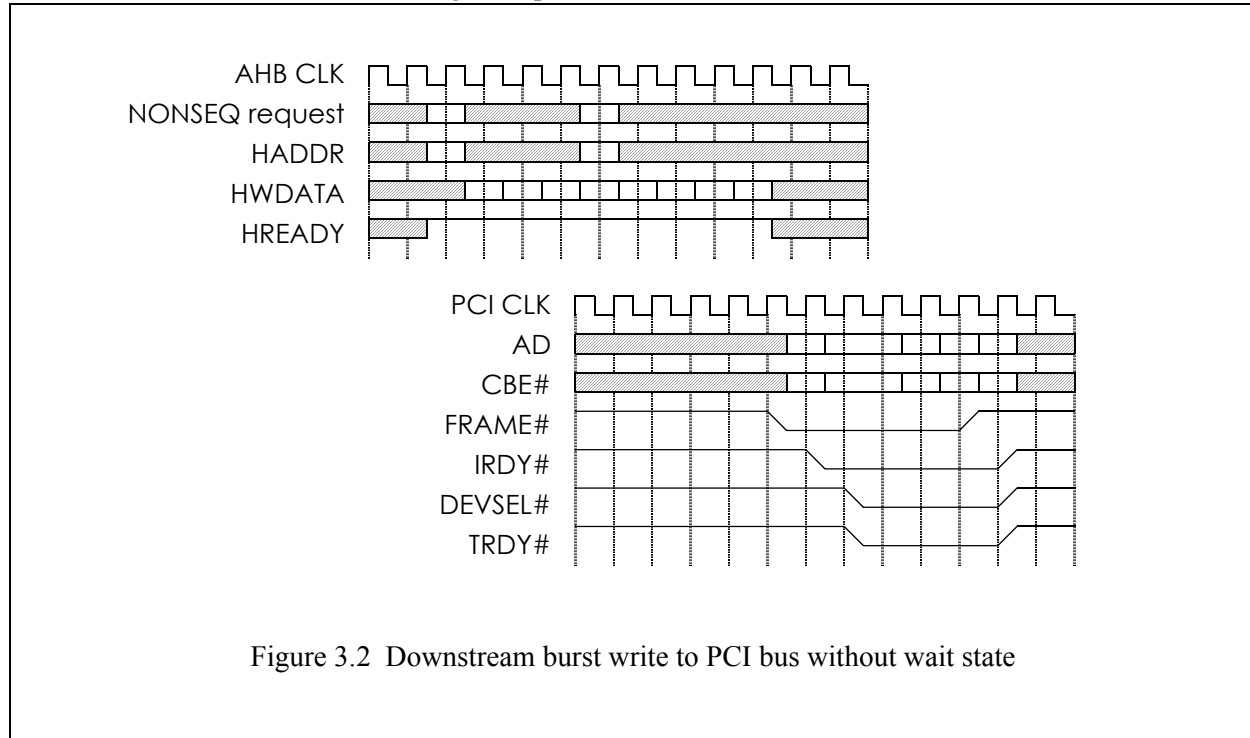


Figure 3.2 Downstream burst write to PCI bus without wait state

The above diagram shows a write request posted in the AHB bus being forwarded to the PCI bus. Data is first received by AHB-PCI host bridge. It asserts HREADY-OUT to receive the write data from the AHB bus master. Data is written to the AHB-PCI host bridge with zero wait state.

After data is posted, the request is then forwarded to the PCI bus interface of the AHB-PCI host bridge. It requests for bus ownership on the PCI bus. As soon as bus ownership is granted, it starts data transfer by asserting FRAME# on the PCI bus and drives the write address and write command on the AD and CBE# signals. One cycle after FRAME# is asserted, the AHB-PCI host bridge asserts IRDY# to start data transfer.

The receiving PCI target asserts DEVSEL# to signal the request is received. It also asserts TRDY# to transfer each words of write data. Write data is read from the posted write buffer on to the PCI bus each time TRDY# is asserted by the target device. Since the AHB-PCI host bridge contains all the write data in its buffers, it does not insert master wait state on the PCI bus. This allows the host bridge to maximize data bandwidth on the PCI bridge.

When the last data is transferred, the AHB-PCI host bridge de-asserts FRAME# to signal the last data. The PCI bus returns to idle state after the last data is transferred.

The AHB-PCI host bridge follows the PCI bus protocol when transfer data on the PCI bus. The target device is allowed to assert PCI wait states during transfer. Tar-



get terminations such as retry, disconnect and abort are also supported by the AHB-PCI host bridge. These types of termination will be described in later sections.

The above diagram shows two write transaction on the AHB bus. As soon as the first write transfer is completed on the AHB bus, the AHB-PCI host bridge can accept a second write request. Write data can be posted into the second write buffer at the same the first write request is processed by the AHB-PCI host bridge. The AHB-PCI host bridge contains 4 write buffers so that up to 4 write transfers (up to 64 bytes each) can be posted in the AHB-PCI host bridge at the same time.

3.5 PCI Bus Write with Wait State

The following examples shows another example burst write of 4 data words. Only one write request ion the AHB bus is shown and multiple wait state is inserted by the target on the PCI bus

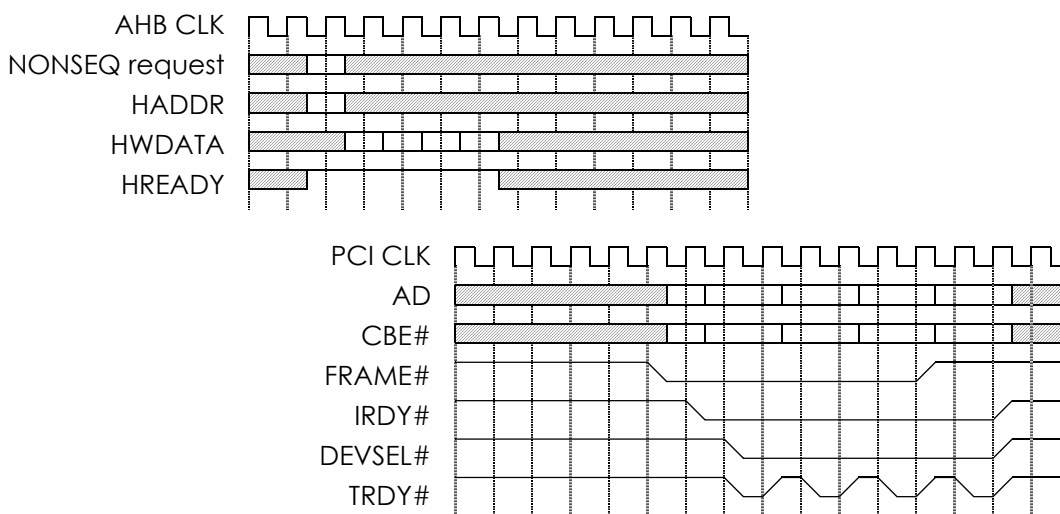


Figure 3.3 Downstream burst write to PCI bus with wait state

No shown in the above diagram is that the AHB-PCI host bridge accepts all target responses as required by the PCI bus specification. Different DEVSEL# speed, target retry and target disconnect are all handled by the AHB-PCI host bridge automatically. Target abort is also signaled to the system through configuration registers. Please see later sections for more detail.



3.6 Delay Read Issued by AHB bus

Read from the AHB bus to the PCI bus is handled as delay read request. The read request is responded with retry by the AHB-PCI host bridge as soon as the request is decoded. This prompts the requestor on the AHB bus to arbitrate for the AHB bus and re-issue the read request later. At the same time, the AHB-PCI host bridge starts the read request on the PCI bus. When the read data is returned from the user interface port, it is stored internally in its read buffer. The read data is identified by the read address.

While the AHB-PCI host bridge is reading the data from the user port, the AHB bus master may re-issue the read request again but each re-issue of the request is retry-ed by the AHB-PCI host bridge until read data has returned from PCI bus.

When the master re-issues the read request after the read data has returned and stored internally, the AHB-PCI host bridge responds to the read request with valid data and asserts HREADYOUT for each data transfer. The AHB-PCI host bridge compares the request address with that of the read data to ensure that the read data is for the same read address.

During the time the master waiting for re-issue the read request, other masters can still write to the AHB-PCI host bridge and post write data into the write buffers.

The following diagram shows a read access to the PCI bus is retry-ed by the AHB-PCI host bridge while the read is in progress on PCI bus. Please note that the requester may re-issue the read access multiple times while data is read from the PCI bus. Each read attempt is terminated with retry until the read data has



returned.

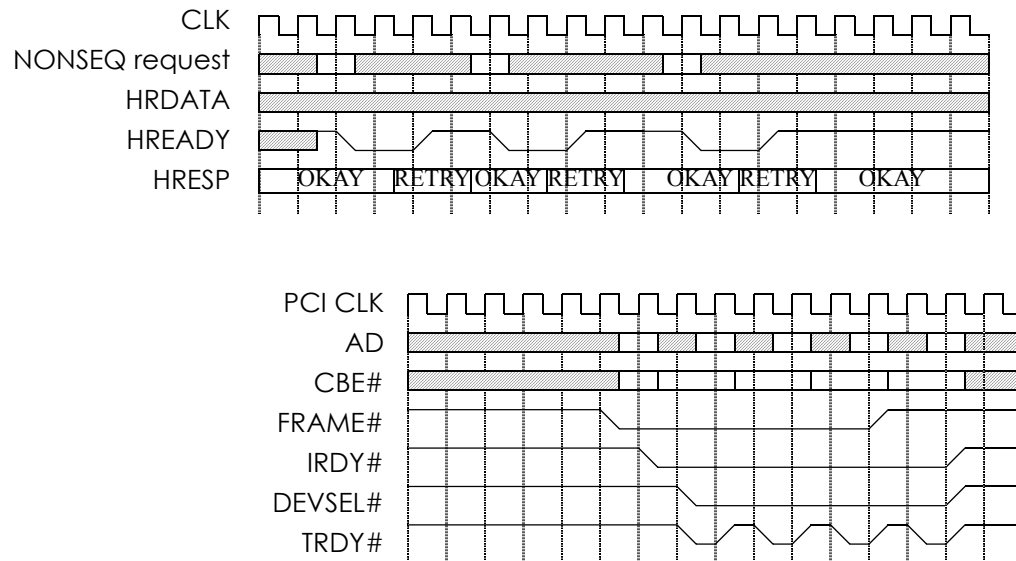


Figure 3.4 Delay read to PCI with retry-ed on AHB bus

The following diagrams shows that after the data is read from the PCI bus, the



next read request to the same address will be responded with the read data.

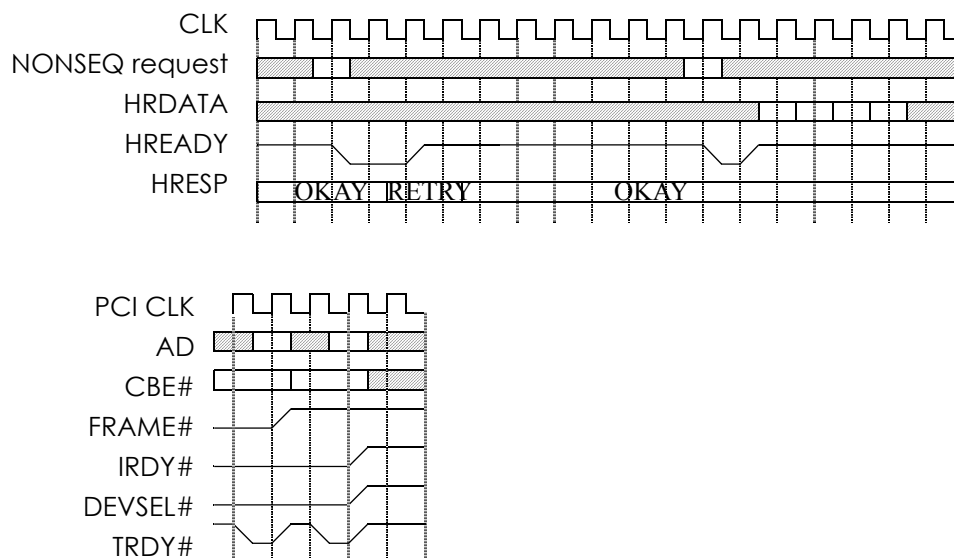


Figure 3.5 Read from AHB bus after data become available

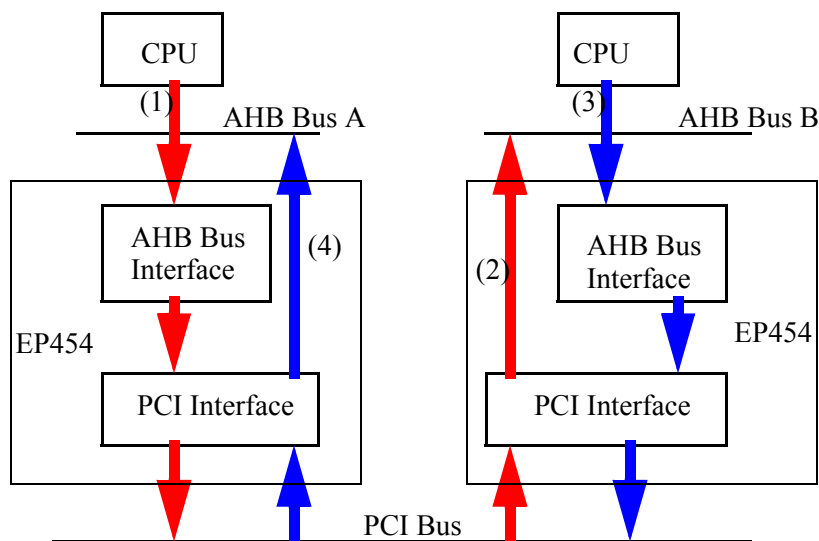
3.7 DeadLock Prevention

It is necessary that all read access targeted to PCI bus are processed as delay read. Without delay read, a bus deadlock may occur between the PCI bus and the AHB bus.

The use of delay read in the AHB-PCI host bridge also conformed with the system transaction ordering requirement of the PCI bus specification. This design is compliant to the “Producer - Consumer Ordering Model” of the PCI bus specification. The following diagram illustrate one such system with two CPU running in paral-



lel.



Legend

1. CPU A reads a data on the PCI bus (red arrows down).
2. The read request on the PCI bus is forwarded to the AHB bus of CPU B. (red arrows up)
3. At the same time CPU B reads a data from the PCI bus (blue arrows down)
4. The read request on the PCI bus is forwarded to the AHB bus of CPU A. (blue arrows up)

If the AHB bus interface on both AHB bus A and B are inserting wait states and waiting for the return data, neither read access (2) or (4) can be completed and results in deadlock.

If either the AHB bus slave device on AHB bus A or B is using delay read. One of the AHB bus will be free to process the read request from the other CPU. As soon as one read is completed, the remaining read can also be completed.

Please note that deadlock situation not only apply to multi-processor systems. Any system with a master/target device on the PCI bus has the potential to cause deadlock unless delay read is used.

Figure 3.6 Bus Deadlock Prevention



3.8 Read Data Invalidation

Read data from the PCI bus are stored into the read buffer internal to the AHB-PCI host bridge. Once data is stored, it is marked as valid data and the address of the data is also stored internally for address comparison.

When a new read request is received, the read address is compared with the stored address for the read data. If the address matches and the data is marked as valid, the read is considered a read hit. The read data is returned to the AHB master in the following clock cycle. Up to 64-consecutive bytes of data can be stored in the read buffer.

After data is marked as valid in the read buffers, there are different conditions when the data is invalidated:

- (1) The read data is returned to the master in a read hit.
- (2) The read address falls within the write address of a write access that is initiated after the read access.
- (3) The number of read misses exceed 16 after the data is marked as valid.

3.9 Matching Data Width

The PCI bus is 32-bit wide while the AHB bus interface supports the AHB HSIZE of 8, 16 and 32-bit. The read and write buffer in the AHB-PCI host bridge are used to match the differences in AHB bus and user interface data width.

For write transfers, data can be written into the write buffers at 8, 16, or 32-bit width based on AHB data width. The AHB-PCI host bridge arranges data into 32-bit wide and aligns them according to HADDR before transferring to the PCI bus. The byte enable signals indicate the byte(s) to be written.

For read transfers, data are read from the user interface at 32-bit width. They are stored in read buffer and can be returned to the AHB. Data is read from the read buffer at the rate as dictated by the AHB data width of the transfer.

3.10 Auto-restart

Data transfer on the PCI bus can be interrupted in many ways. For example, a target may initiate a disconnect during a burst transfer for any reason. The AHB-PCI host bridge may also terminate a transfer due to time out before the burst sequence is completed.

If any of these conditions occurs before the intended number of data transfer has completed, the AHB-PCI host bridge will terminate the PCI cycle according to the PCI specification and will then automatically re-initiate the transfer request on the PCI bus.

Auto-restart is transparent to the AHB bus device who initiated the transfer. Both read and write accesses can be re-started if they are interrupted on the PCI bus. The following diagram shows target disconnect during read and restarted by the



AHB-PCI host bridge.

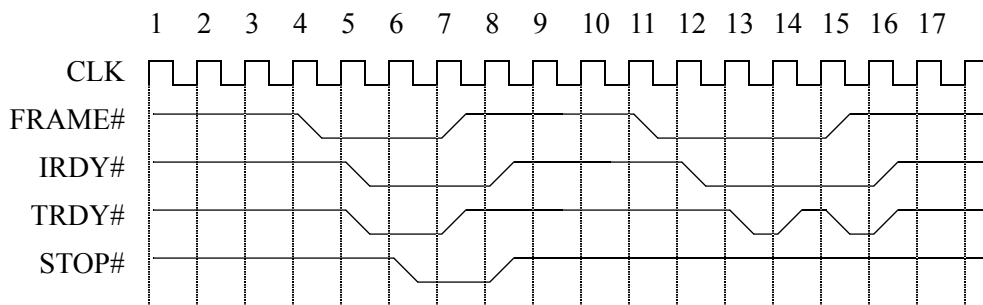


Figure 3.7 Master burst read with auto restart

3.11 Target retry control

With auto-restart, the master will continuously re-issue the request if the target keep responding with target retry. The EP454 allows the user further control of handling target retry response. When this feature is enabled, the EP454 core impose a limit on how much target retry is considered as reasonable for each access from the AHB bus. If the target issue too many target retry for one AHB access, the target is considered defective and the core aborts the request after the number of target retry exceed the allowed limit. The core implements the following registers in the configuration register area to support this feature.

Retry counter - This is a 16-bit register that specifies the maximum number of auto-restart the PCI master will issue for each access request. If the external target issues retry that exceed this value, the master will assume that the external target has failed not retry the request again. A new AHB request, however, can be issued to this target.

Counter enable. Bit 0 of this register indicates if the retry counter is enabled. If this bit is “0”, the retry counter is not enabled and the PCI master does not impose any limit in the number of auto-restart due to retry. If this bit is “1”, the retry counter limit is imposed. Bit 7 to 1 of this register are not used.

Retry overflow flag. Bit 0 if this register is set if the master abort a request due to retry counter limit. User can clear this bit by writing “1” to bit 0. Writing “0” has no effect. Bit 7 to 1 of this register are not used.

3.12 Target abort

There are several PCI mechanism for the target device to terminate data transaction other than normal termination. They are target retry, disconnect and abort.



When the target signals retry or disconnect, the AHB-PCI host bridge automatically restart the transaction as explained in previous section.

If the target signals a target abort, or if no target response to the request after 6 PCI clock cycles, the AHB-PCI host bridge immediately terminates the transaction. This condition is signaled by the AHB-PCI host bridge by setting the target abort bit or the master abort bit in the PCI status register. The user may use these status bits to trigger interrupt on the system. Please consult Eureka Technology if such function is needed.

If the master or target abort is due to a AHB to PCI post write, no additional signaling to the AHB bus is performed. If the master or target abort is due to a AHB to PCI read, data returned to the AHB bus is signaled as error by the AHB interface. In both read and write cases, the PCI status bits is available as output pins from the core.

3.13 Byte enable

Bytes within the 32-bit data width of the PCI bus can be read or written individually. The byte enable signal to the PCI bus is computed based on the AHB bus data width and burst size.

When a burst read is requested, the AHB-PCI host bridge ensures that all bytes are enabled for read.

3.14 Burst Sequence

Burst sequence is the address order in which successive data is accessed. The PCI host bridge supports linear increment burst order.

When memory address is accessed by the host bridge, the two least significant bits of the PCI address are set to zero by the AHB-PCI host bridge to indicate linear increment burst order. The two least significant bits are not forced to zero when I/O space is accessed.

3.15 Configuration cycles

One of the main difference between the host bridge and master/target is the ability to initiate PCI configuration cycles. The CPU requests configuration access on the PCI bus by writing or reading from the CONFIG_ADDR and CONFIG_DATA registers. These two registers are contained in the AHB-PCI host bridge host bridge. The CONFIG_ADDR and CONFIG_DATA registers can be accessed by specifying HSEL[2] and HSEL[3] on the AHB bus, respectively.

To issue configuration access on the PCI bus, the user logic or the CPU should execute the following steps:

Step 1. Write the proper address to the CONFIG_ADDR register. To access this register, asserts HSEL[2] during the AHB slave access, address is don't



care. The content of the CONFIG_ADDR register is described below. The CONFIG_ADDR register should always be accessed as a 32-bit register.

Step 2. Read or write the CONFIG_DATA register. To access this register, asserts HSEL[3] during the AHB slave access, address is don't care. The core will issue a configuration read or write to the location specified in the CONFIG_ADDR register.

3.16 CONFIG_ADDR and CONFIG_DATA registers

The configuration mechanism implemented by the host bridge is the PC-compatible standard mechanism, defined by the PCI specification as Configuration Mechanism #1. Both type 0 and type 1 configuration cycles are supported. This mechanism requires the host bridge to decode the CONFIG_ADDR value and generates special PCI configuration register. This function is contained in the host bridge.

Type 0 and type 1 configuration cycle implies that the host bridge is capable of configure PCI agents in the same bus segment as well as PCI agents in the other side of a PCI-to-PCI bridge. In other words, multiple segments PCI bus is supported by the host bridge.

To initiate a PCI configuration access, the CPU is first required to write the address into the CONFIG_ADDR register. The format of the CONFIG_ADDR register is the following:

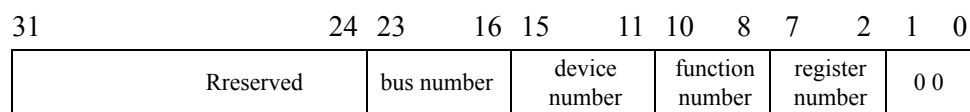


Figure 3.8 CONFIG_ADDR format

The AHB-PCI host bridge translates the CONFIG_ADDR information into a configuration register address based on whether it is a type 0 or type 1 configuration access. The address translation method is defined in PCI specification. The following diagrams show type 0 and type 1 address translation between CONFIG_ADDR register and PCI address. Type 0 translation is used if the con-



figuration target is local (bus number = 0), otherwise type 1 translation is used.

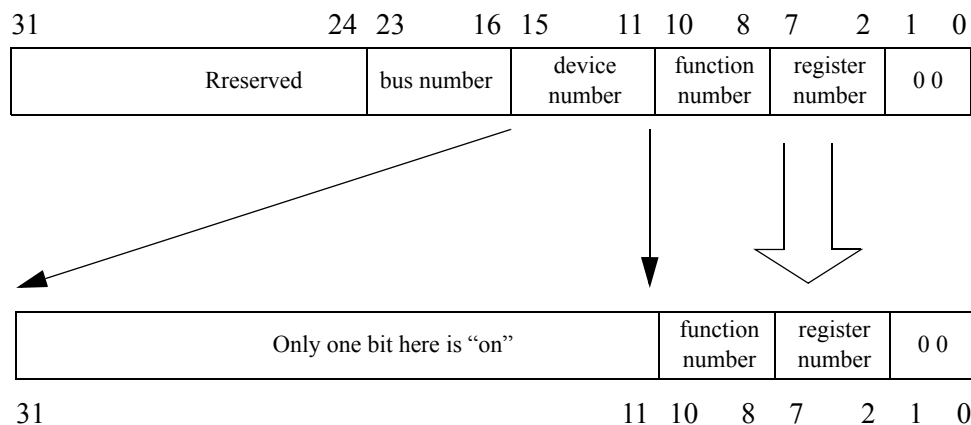


Figure 3.9 Type 0 translation

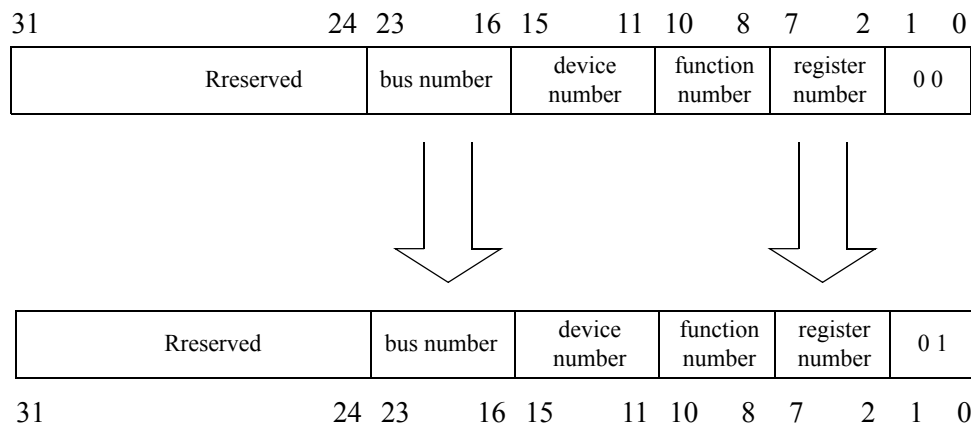


Figure 3.10 Type 1 translation

The CPU then reads or writes to the CONFIG_DATA register to initiate a configuration read or write access. The read/write access to the CONFIG_DATA triggers the AHB-PCI host bridge to initiate PCI configuration access to the PCI bus. If it is a read, configuration data read from the PCI bus is deposited to the CONFIG_DATA register and returned to the AHB bus. Delay read is used in CONFIG_DATA and CONFIG_ADDR read as in normal read access. If it is a write access, data written into the CONFIG_DATA register is written to the PCI bus as configuration write data. Write posting is used in CONFIG_DATA write as



in normal write access. from AHB bus.



4. PCI Target Functional Description

When forward transfer request upstream (from the PCI bus to AHB bus), the AHB-PCI host bridge functions as a PCI target device. It is mapped to PCI memory or I/O space through the regular base address register mechanism as defined in PCI specification. The request to the target is forwarded to the AHB bus. Upstream request allows other PCI devices on the bus to access system memory and other system resources that resides on the AHB bus.

4.1 Upstream Write Posting

The AHB-PCI host bridge contains 2 write buffers for posting write data as PCI target. These two write buffers operate in parallel so that write data can be posted into one write buffer from the PCI bus while data is read from the other write buffer to transfer upstream. Each write buffer can store up to 64 bytes of data.

When the AHB-PCI host bridge detects PCI write request that matches its own address as defined in the PCI base address register, it claims the write access by asserting DEVSEL# signal in the PCI bus. The AHB-PCI host bridge functions as a target for this transfer.

If either write buffer is available, The AHB-PCI host bridge asserts TRDY# to transfer data. Burst write data can be transfer with no wait state in between data.

If both write buffers are occupied, the AHB-PCI host bridge asserts STOP# to respond to the transfer with PCI retry. In this event, the PCI master will arbitrate for the bus again to re-issue the transfer. The AHB-PCI host bridge accepts the write data when either write buffer becomes available.

After write data is posted in the write buffer, the AHB-PCI host bridge forwards the request upstream to the AHB bus. Because the AHB bus and the PCI bus operate in different clock domains, asynchronous request/acknowledge protocol is used to signal transfer across the clock domains. This protocol is the same as the one used in the downstream direction.

When the request reaches the AHB bus clock domain, the AHB-PCI host bridge arbitrates for the AHB bus and initiate write request on the AHB bus.



The following diagram illustrate a write access from PCI bus to AHB bus.

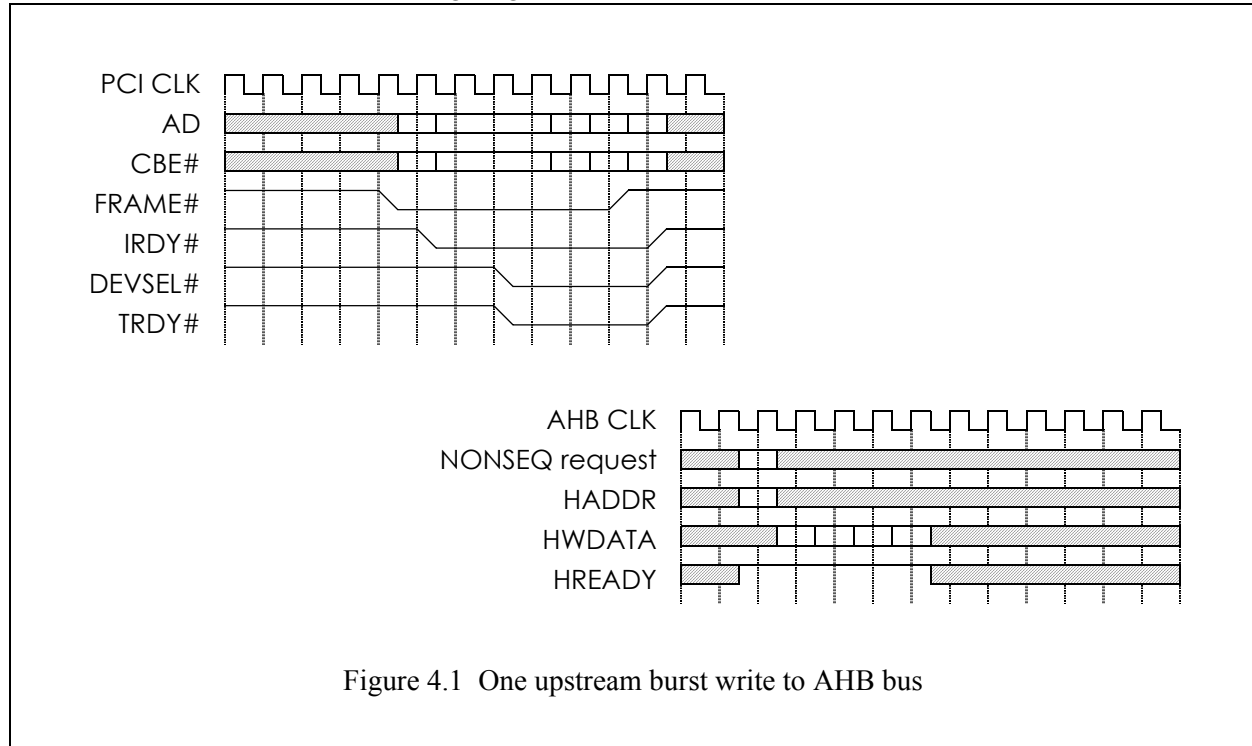


Figure 4.1 One upstream burst write to AHB bus

The following diagram illustrate a second write access is posted on the PCI bus while write operation on AHB bus is in progress.

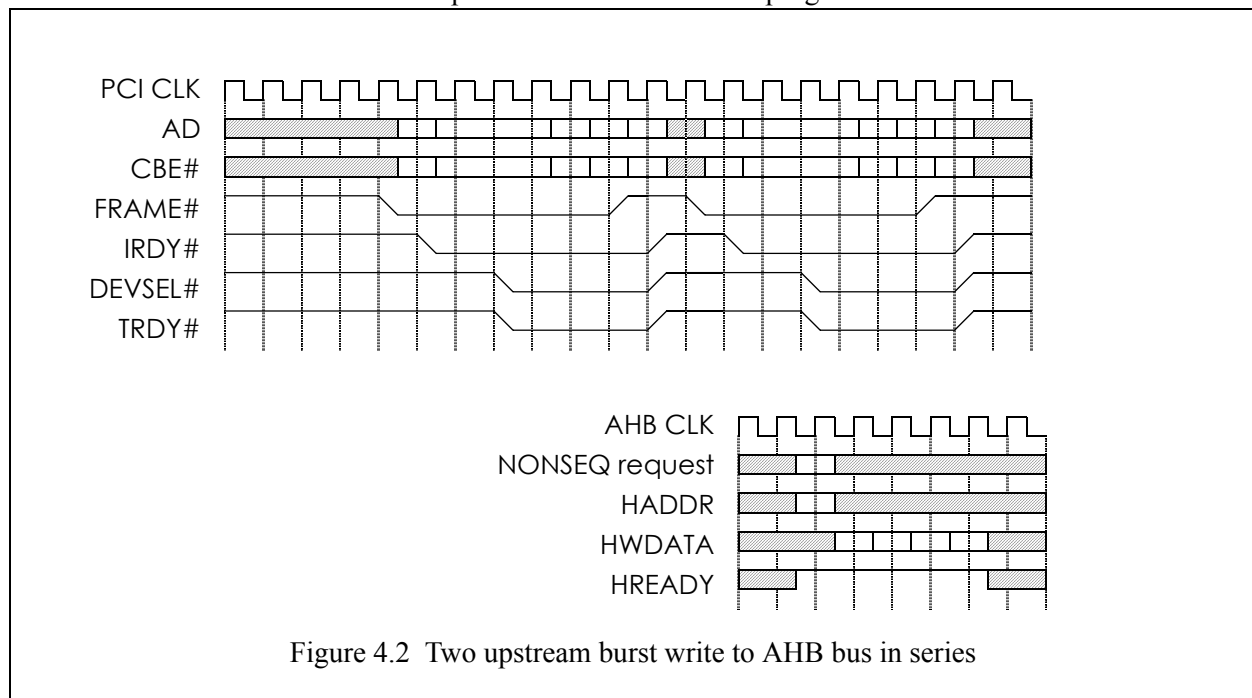


Figure 4.2 Two upstream burst write to AHB bus in series



4.2 Upstream Read Request

Unlike downstream read request, upstream read request from PCI to AHB bus is processed as real-time read. When the AHB-PCI host bridge detects PCI read targeting AHB bus, it claims the cycle by asserting DEVSEL# signal and forward the request upstream to the AHB bus clock domain. Read access on the AHB bus is initiated to retrieve the read data. During this time, the AHB-PCI host bridge inserts wait state on the PCI bus to until read data is available from the AHB bus.

The AHB-PCI host bridge accepts both single and burst access from the PCI bus. The AHB-PCI host bridge accepts burst access transfer based on the PREFETCH_LIMIT input and the PREFETCHABLE bits in the base address register. If the PREFETCHABLE bit is zero, prefetch read is not allowed and the core read one word only and then issues disconnect on burst read. If PREFETCHABLE bit is one, the core prefetch up to the word boundary as specified by the PREFETCH_LIMIT input. Typically the core can be set up to prefetch up to the next 4, 8 or 16-word boundary. When the master attempts to burst across the specified word boundary, the AHB-PCI host bridge issues disconnect so that the master will issue a separate access starting from the next 16-word boundary.

Once burst read is initiated by the PCI bus master, the AHB-PCI host bridge issues burst read on the AHB bus until either the word boundary is reached or when the master terminates the read request.

The following diagram shows an upstream read request.

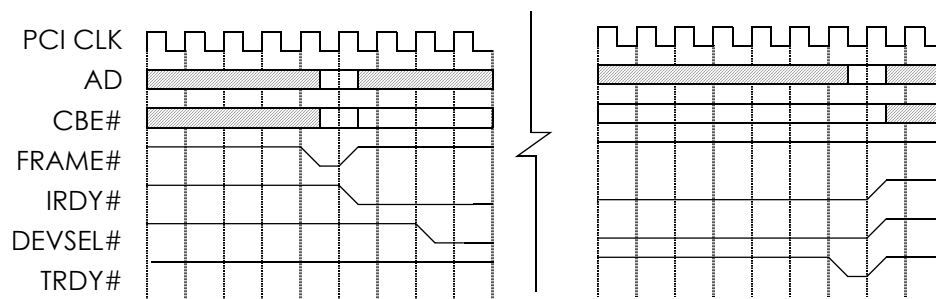


Figure 4.3 Upstream single read to AHB bus

4.3 Concurrent master/target operation

The AHB-PCI host bridge supports concurrent bus master and target operation. An AHB bus device may issue downstream request to the PCI bus. While the master operation is in progress within the AHB-PCI host bridge, another PCI bus



master may have ownership of the PCI bus and issue an upstream request to the AHB bus. The host bridge allows concurrent function since it has dedicated resources for both the upstream and downstream requests.

The host bridge does not issue wait state on the AHB bus due to PCI bus response. It always responds to AHB bus request within finite number of clock cycles. If a downstream request is concurrent with an upstream read request, the host bridge issues wait state on the PCI bus, thus preventing the downstream operations to complete. Since the AHB bus operation terminates within finite number of clock cycles, the upstream read request will be able to complete, releasing the PCI bus, and allowing the downstream request to go through.

If a downstream request is concurrent with an upstream write request. The host bridge will be able to post the upstream write request into its upstream write buffer, release the PCI bus, and allow the downstream request to complete.

4.4 Request ordering

The AHB-PCI host bridge maintains the request order within each direction that bus request is forwarded. Write operation from each bus is posted into the write buffer according to the order of the request in the originating bus. The same order is followed when the request is issued on the destination bus. The order is also maintained between read and write requests.

The AHB-PCI host bridge does not guarantee the request order if the requests are originated in two different buses. For example, one request originated from the PCI bus and another request originated from the AHB bus at approximately the same time may complete in any order relative to each other in their destination buses.

4.5 Parity

The AHB-PCI host bridge generates the PCI parity signal, PAR, whenever it drives the PCI AD bus. If the parity enable bit in the configuration register is enabled, the host bridge will also detect and signal bus parity error whenever it receives data from the PCI bus. Data parity error is signaled by asserting the PERR# signal. However, the AHB-PCI host bridge continues to process the bus transaction even when parity error is detected.

Address parity error is also detected if the system error control bit in the configuration is enabled. The host bridge signals address parity by asserting the SERR# signal. Even when address parity is detected, if the host bridge is the selected device, it claims the cycle and terminates as though the address was correct.

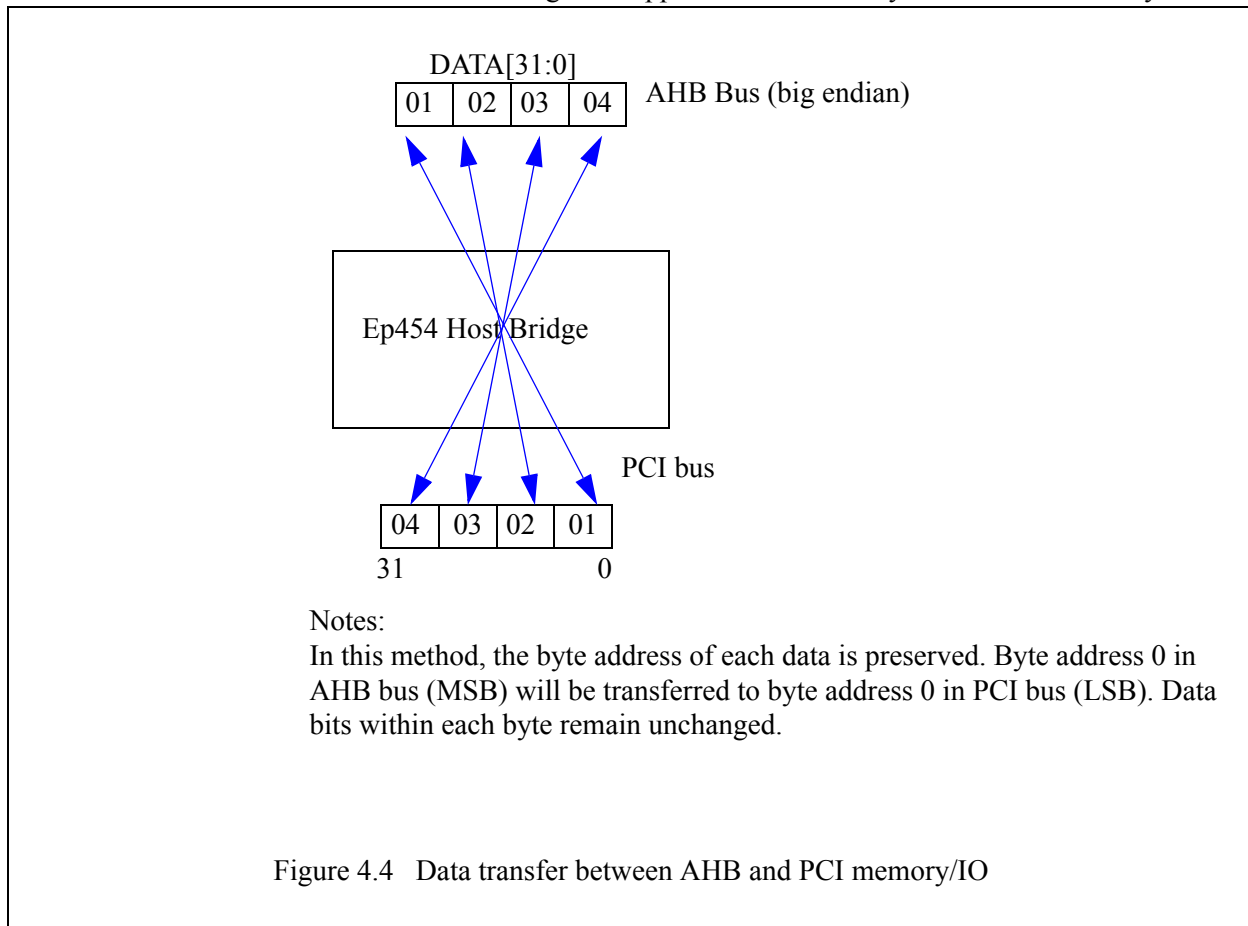
The system handling after PERR# and SERR# detection is very dependent on overall system design. The AHB-PCI host bridge can assist system error handling by making the PERR# and SERR# status bits of the configuration register visible to system logic according to user specification.



For details on PCI parity generation and detection, please refer to the PCI specification. The host bridge, however, handles all parity generation and detection task so it is transparent to the local bus.

4.6 Big Endian

The host bridge supports both big endian and little endian on AHB bus. The `BIG_ENDIAN` input pin indicates which mode the AHB bus operates on. The PCI bus is always in little endian mode so if the AHB bus is in big-endian mode, the core converts the position of each byte according to the following diagram. Please note that the following table applies to PCI memory and IO transfers only.



During access to the `CONFIG_ADDR` and `CONFIG_DATA` registers (configura-



tion access, the EP454 core converts big-endian data in the following format:

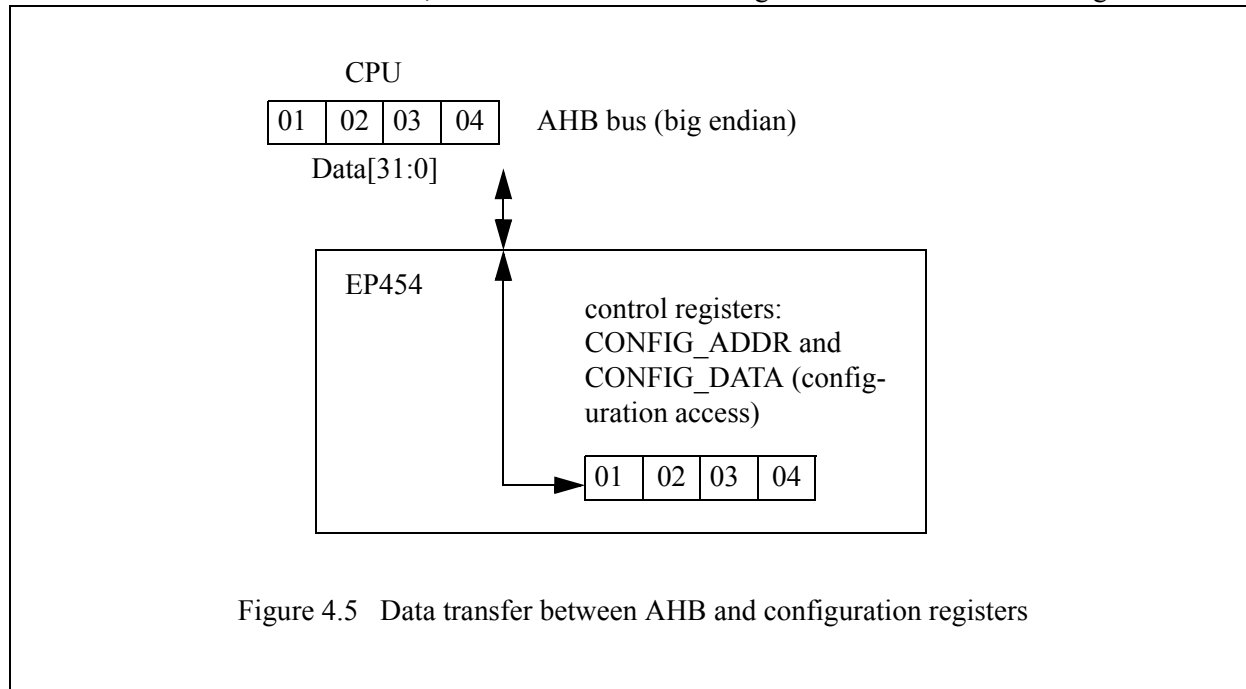


Figure 4.5 Data transfer between AHB and configuration registers

4.7 Interrupt

The EP454 supports interrupts generated by the user logic and propagate the interrupt to the PCI bus. When the user interface request interrupts, it asserts H_INTA# and the master/target asserts INTA# at the same time. INTA# is an open drain output according to PCI specification.

In addition, the master/target contains the proper interrupt information in the PCI configuration register as required by the PCI specification. The register are in location 3C(Hex) and 3D(Hex). Please see the configuration register section for more detail. The INTERRUPT_PIN input should be 01(hex) to indicate usage of INTA# signal.

When the core operates at PCI spec 2.3 mode, (selected by SEL_23 input pin), the core offers additional control for PCI interrupt. Bit 10 of the command register serves at the interrupt disable pin and bit 3 of the status register serves as the interrupt status bit. Please see the control register section for more detail.

4.8 Transaction type

The host bridge supports different transaction commands as bus master and as target. As a bus master it can initiate the following transactions:

- Memory Read
- Memory Write



IO Read
IO Write
Configuration Read
Configuration Write

As a bus target, it can receive the following transactions:

Memory Read
Memory Read Line
Memory Read Multiple
Memory Write
Memory Write & Invalidate
IO Read
IO Write
Configuration Read
Configuration Write

4.9 Self configuration

The host bridge contains configuration registers as any PCI agent. The configuration registers can be accessed by the CPU in the same way it accesses the configuration registers of other PCI devices. When a configuration request is targeting the host bridge's own configuration registers, the host bridge functions as both the master and the target for the configuration cycle on the PCI bus.

The host bridge can be mapped to any PCI device number as long as IDSEL is connected to the proper AD line. In typical configuration, the host bridge is mapped as device 0 on the bus.

4.10 Configuration Registers

The AHB-PCI host bridge features all the configuration register required by the PCI specification. The following is a map of the configuration register. All configuration register accesses are processed automatically by the host



bridge and does not propagate into the AHB bus.

Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1				14h
Base Address Registers 2				18h
Base Address Registers 3				1Ch
Base Address Registers 4				20h
Base Address Registers 5				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved				34h
Reserved				38h
Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch
Retry over-flow flag	Counter enable	Retry counter		40h

Figure 4.6 Configuration Registers

The following section describes each configuration register bits in detail:

00h: Device ID - Read-only field defined by the user.

Vendor ID - Read-only field defined by the user.

04h: Command Register:

Bit	Description	Default
15-11	Reserved	Read only, 0
10	This bit is used only in PCI spec 2.3. If this bit is 0, assertion of INTA#-INTD# is allowed. If this bit is 1, assertion of INTA#-INTD# is not allowed. This bit is hardwired to "0" in PCI spec 2.2	Read/write (2.3) Read only, 0 (2.2)
9	Fast back-to-back, read only unless requested by user	Read only, 0



Bit	Description	Default
8	SERR#	Read/write
7	Address data stepping, not needed	Read only, 0
6	Parity error response	Read/write
5	VGA palette snooping, read only unless requested by user, read only unless requested by user	Read only, 0
4	Master MWI enable.	Read/write
3	Special cycle response, read only unless requested by user	Read only, 0
2	Bus master	Read/Write
1	Memory enable	Read/Write
0	IO enable	Read/Write

06h:Status Register

Bit	Description	Default
15	Parity error	Read/Clear
14	SERR#	Read/Clear
13	Master abort	Read/Clear
12	Target abort as master	Read/Clear
11	Target abort as target	Read/Clear
10-9	DEVSEL timing	Read only, 10
8	PERR#	Read/Clear
7	Fast back-to-back, read only unless requested by user	Read only, 1
6	UDF, read only unless requested by user	Read only, 0
5	33/66Mhz, read only, specified by the user using the run_66mhz input.	Read only, 1/0
4	Capability list. This bit is normally 0 unless optional features such as power management or hot swap is implemented	Read only
3	This bit is used in PCI spec 2.3 only. This bit reflect the inverted value of H_INTA# input. INTA# is asserted only when this bit is "1" and interrupt disable bit command register is "0". In PCI spec 2.2 this bit is always "0".	Read only
2-0	Reserved	Read only, 0



08h: Class Code - Read-only field defined by the user.

Revision ID - Read-only field defined by the user.

0Ch: Cache line size - All bits are read/writable The core supports cacheline size of 4, 8 and 16 words.

0Dh: Latency Timer - All bits are read and write.

0Ch: Header- All bits are read-only and zero unless requested by user.

0Ch: BIST - All bits are read-only and zero unless requested by user.

10h-24h: Base Address Register - The number of read-write bits and read-only bits are defined by the user using the BAR_SETUP signals. The BAR_SETUP signals are described in the Signal Description section of this spec. The number of BAR to be implemented can be further specified by the user. Please contact Eureka Technology for detail.

28h: Cardbus CIS Pointer - Read-only and zero

2Ch: Subsystem ID - Read-only and defined by user.

Subsystem Vendor ID - Read-only and defined by user.

30h: Expansion ROM base address. Specified by user and implemented by Company.

34h to 38h: Reserved or not used - Read-only and zero.

3Ch: Interrupt line - read and writable.

3Dh: Interrupt pin - Read-only and defined by user.

3Eh: Minimum Grant - Read-only and zero unless requested by user.

3Fh: Maximum Latency - Read only and zero unless requested by user.

40h: Retry counter - This is a 16-bit register that specifies the maximum number of auto-restart the PCI master will issue for each access request. If the external target issues retry that exceed this value, the master will assume that the external target has failed not retry the request again.

42h: Counter enable. Bit 0 of this register indicates if the retry counter is enabled. If this bit is "0", the retry counter is not enabled and the PCI master does not impose any limit in the number of auto-restart due to retry. If this bit is "1", the retry counter limit is imposed. Bit 7 to 1 of this register are not used.

43h: Retry overflow flag. Bit 0 if this register is set if the master abort a request due to retry counter limit. User can clear this bit by writing "1" to bit 0. Writing "0" has no effect. Bit 7 to 1 of this register are not used.



5. Design Variations

Different design variations of the AHB-PCI host bridge is available. By integrating different functional blocks to the core, additional features such as DMA and SDRAM interface can be realized. The following diagrams show two typical variations of the AHB-PCI host bridge. Please contact Eureka Technology for more detail.

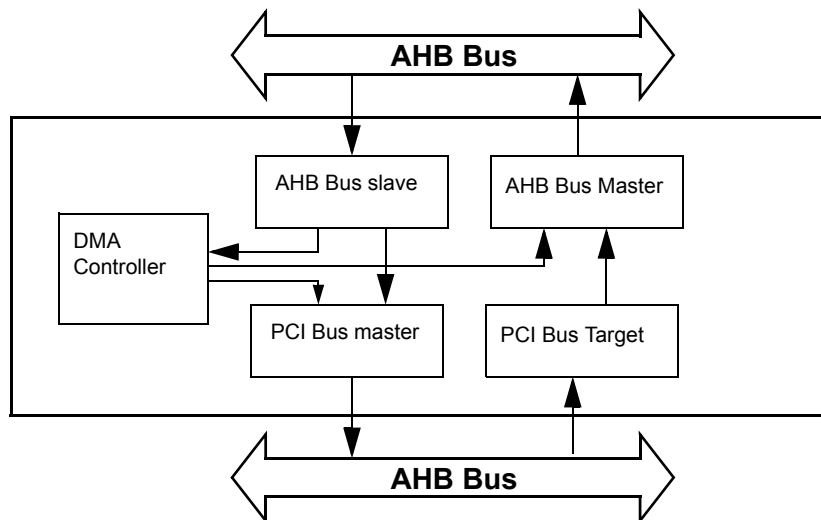


Figure 5.1 Host Bridge with DMA

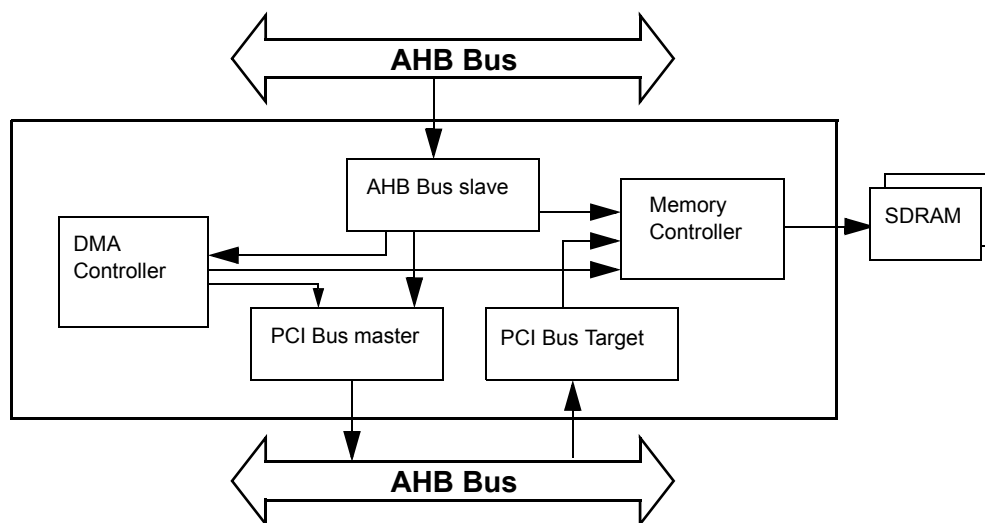


Figure 5.2 Host Bridge with DMA and Memory Controller

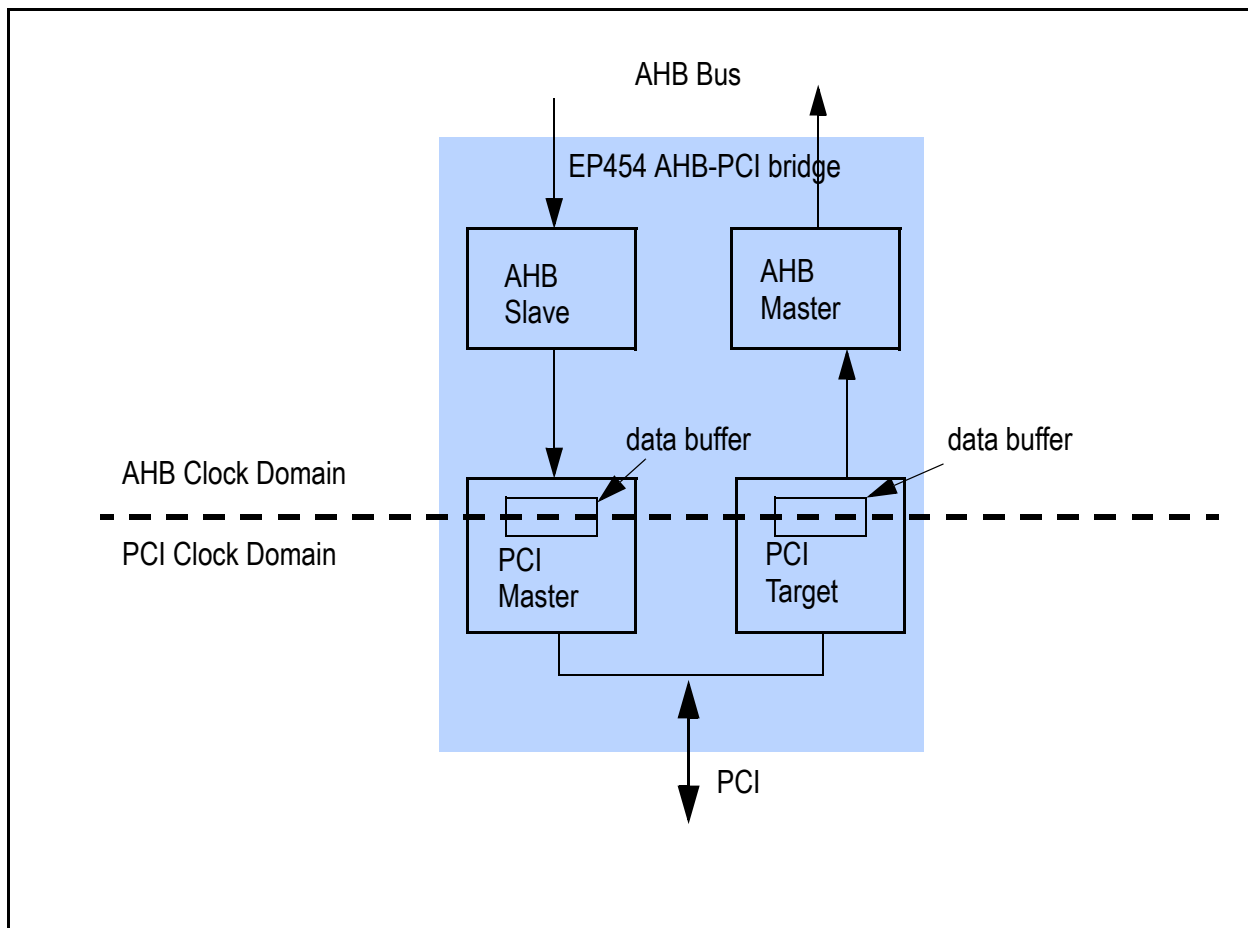


Appendix A. Address Translation Option

This appendix describes the upstream and downstream address translation feature optional to the EP454 AHB-PCI bridge.

Upstream address translation applies when PCI transaction is forwarded to the AHB bus. Downstream address translation applies when AHB transaction is forwarded to the PCI bus.

Address translation is an optional feature of the AHB-PCI bridge. Please contact Eureka Technology if address translation or other customized translation scheme is needed.



A.1 AHB to PCI and IP core mapping

Three different areas can be accessed from the AHB bus through the AHB-PCI bridge. The three areas are:

1. PCI memory space.



2. PCI I/O space.

3. Registers: CONFIG_ADDR register, CONFIG_DATA register, translation and DMA control registers.

The selection of the three areas is controlled by the HSEL input. Three bits are provided in this input and the mapping is:

HSEL	Address Space
HSEL[0]	PCI memory space
HSEL[1]	PCI I/O space
HSEL[2]	All register

PCI memory and I/O access are forwarded directly to the PCI bus. The address can be translated as described in the translation control register section. Access to various registers are either terminated internally or trigger PCI configuration access. No address translation is needed for register access.

A.2 PCI to AHB mapping

It is possible to implement a maximum of 7 areas of PCI address spaces, controlled by six BARs and one expansion ROM BAR. Each of these access can be claimed by the PCI target of the EP454 and forwarded to AHB bus. The size of each BAR is controlled by the user through the BAR_SETUP input pins as described in the AHB-PCI spec. The size of each BAR should be static once the device comes out from reset.

The PCI address of each area controlled by the BAR is software programmable as defined in PCI spec. The PCI address of each area controller by the BAR is allocated by the operating system by writing to the PCI configuration register of the PCI target.

When the PCI target claims a PCI request and forward it to the AHB bus, the PCI address can be translated to a different AHB address using the translation control registers. The detail of AHB translation is described in the translation control register section.

A.3 Address Translation Signals

The address translation scheme requires the following information. The information can be either located in the AHB-PCI bridge or external to the bridge. If they are external to the bridge, all values should be brought to the AHB-PCI bridge as static inputs. User can implement registers outside the bridge as part of their system registers and brought in the values as static inputs to the core. The number of translation register depends on the total number of PCI BAR being implemented.



The following table list the maximum number.

Table A.1: Address Translation Inputs

Static Input	Space	Control Register
A2P_MM	AHB to PCI address transla- tion registers	PCI memory mask
A2P_MA		PCI memory translation address
A2P_IM		PCI I/O mask
A2P_IA		PCI I/O translation address
P2A_0M	PCI to AHB address transla- tion registers	BAR0 mask
P2A_0A		BAR0 translation address
P2A_1M		BAR1 mask
P2A_1A		BAR1 translation address

A.4 AHB to PCI Address Translation

When a transaction is forwarded from the AHB bus to the PCI bus, the AHB address is translated according to the PCI mask registers and the PCI translation address registers.

The PCI memory (or I/O) mask register is a 32-bit register with the 8 LSB hard-wired to “0”. A “1” in each bit indicates the corresponding bit in the AHB address will be replaced with the value in the PCI memory (or I/O) address registers. The valid value of the mask registers are:

8000_0000(hex)
C000_0000(hex)
E000_0000(hex)
F000_0000(hex) to
FFFF_FF00(hex).

A value of F000_0000(hex) in the mask register indicates that the upper 4 bits of the AHB address will be replaced with the upper 4 bits in the translation address register when the request is forwarded to the PCI bus. The lower 28 bits are not changed when forwarded to the PCI bus.

A.5 PCI to AHB Address Translation

When a transaction is forwarded from the PCI bus to the AHB bus, the PCI



address is translated according to the BAR mask registers and the BAR translation address registers.

The BAR mask register is a 32-bit register with the 8 LSB hardwired to “0”. A “1” in each bit indicates the corresponding bit in the PCI address will be replaced with the value in the BAR address registers. The valid value of the mask registers are:

8000_0000(hex)
C000_0000(hex)
E000_0000(hex)
F000_0000(hex) to
FFFF_FF00(hex).

A value of F000_0000(hex) in the mask register indicates that the upper 4 bits of the PCI address will be replaced with the upper 4 bits in the translation address register when the request is forwarded to the AHB bus. The lower 28 bits are not changed when forwarded to the AHB bus.

The usage of both the BAR mask and BAR_SETUP will allow the user to have a range that is smaller than the defined PCI BAR size when mapped to AHB space. For example, a 64kByte PCI BAR could map to a 1kByte space in AHB space on a 1kByte boundary by using the mask value of FFFF_FC00(hex). In this example the 1kByte space would alias across the full 64kByte BAR space.

A.6 Translation Enable/Disable

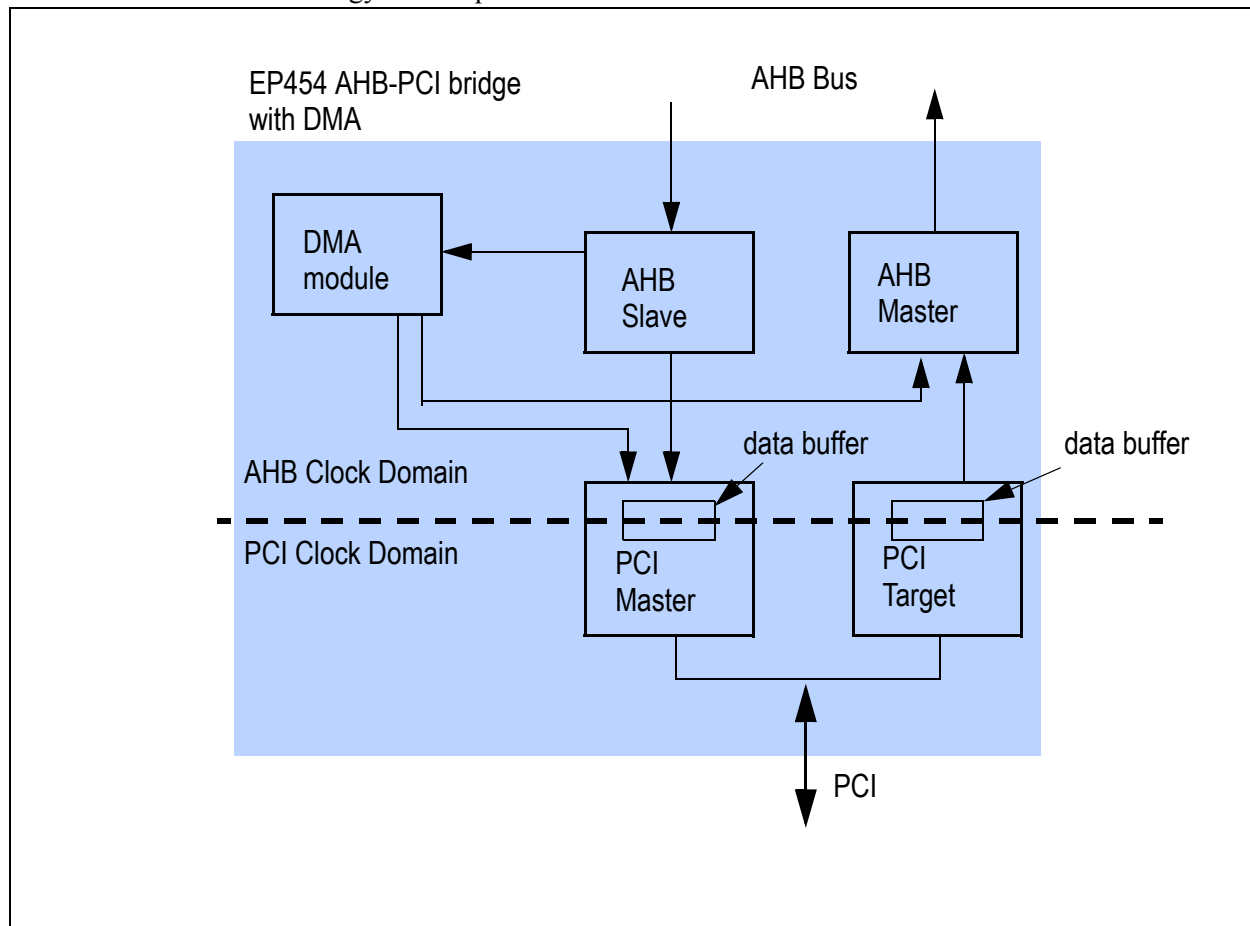
When all bits in the mask register are “0”, address translation for the corresponding space is disable.



Appendix B. DMA Controller

This appendix describes the optional DMA feature of the EP454 AHB-PCI bridge. The DMA function is contained in a module that can be integrated with the EP454. Once programmed, the DMA controller is capable of transferring large amount of data between the AHB and PCI bus without CPU assist.

DMA is an optional feature of the AHB-PCI bridge. Please contact Eureka Technology if this optional feature is needed.



B.1 Features

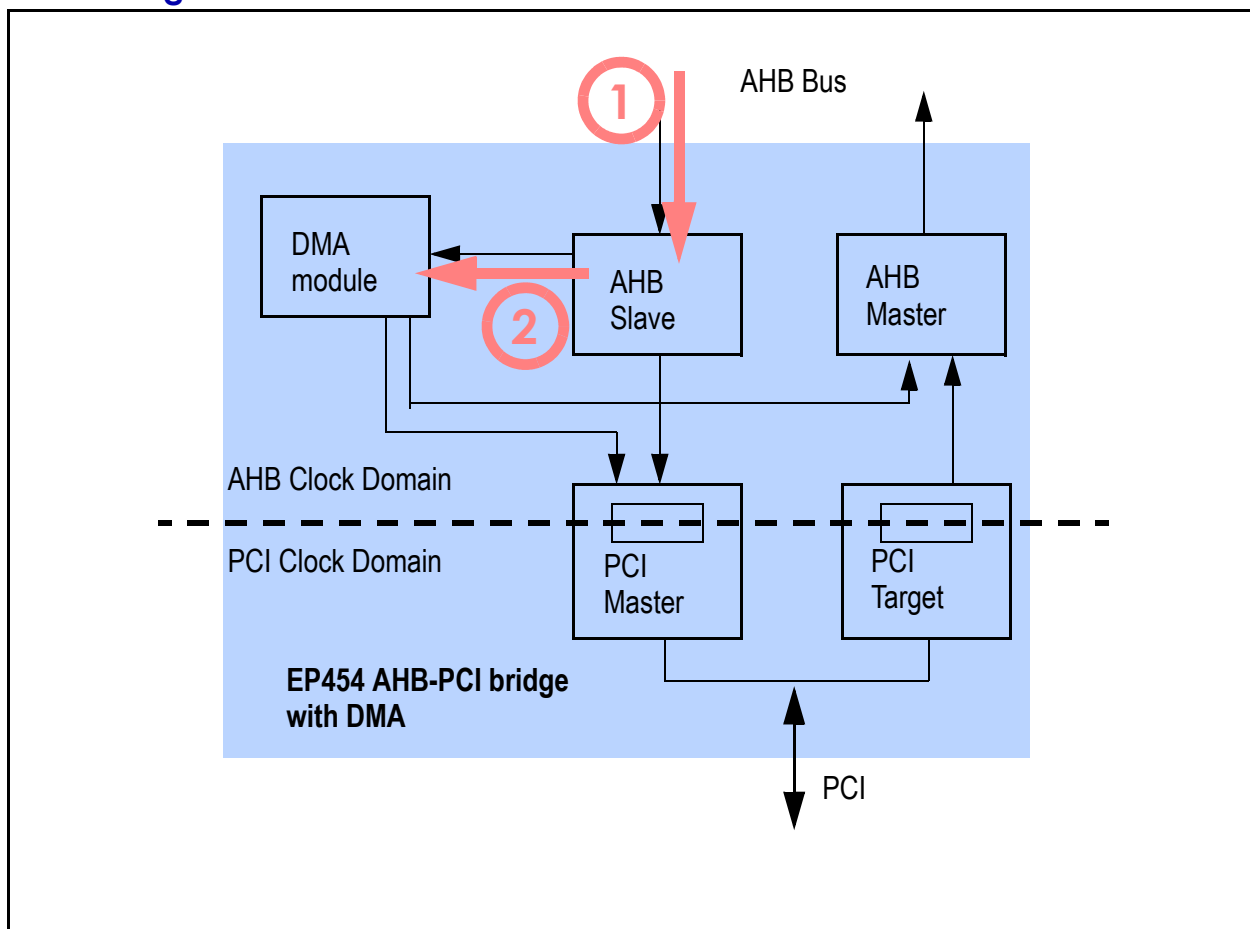
- Multiple independent DMA channels.
- Designed for ASIC or FPGA implementations in various system environments.
- Transfers data between AHB and PCI bus.
- Supports scatter-gather with DMA linked list.
- Supports both hardware initiated transfer and software initiated transfer.

- Supports burst transfer to maximize data bandwidth.
- Automatic address increment.
- Interrupt generation on transfer completion.

B.2 Operation Diagrams

Within this section, different access of the AHB-PCI bridge DMA controller are described step-by-step.

B.2.1 Program the DMA controller

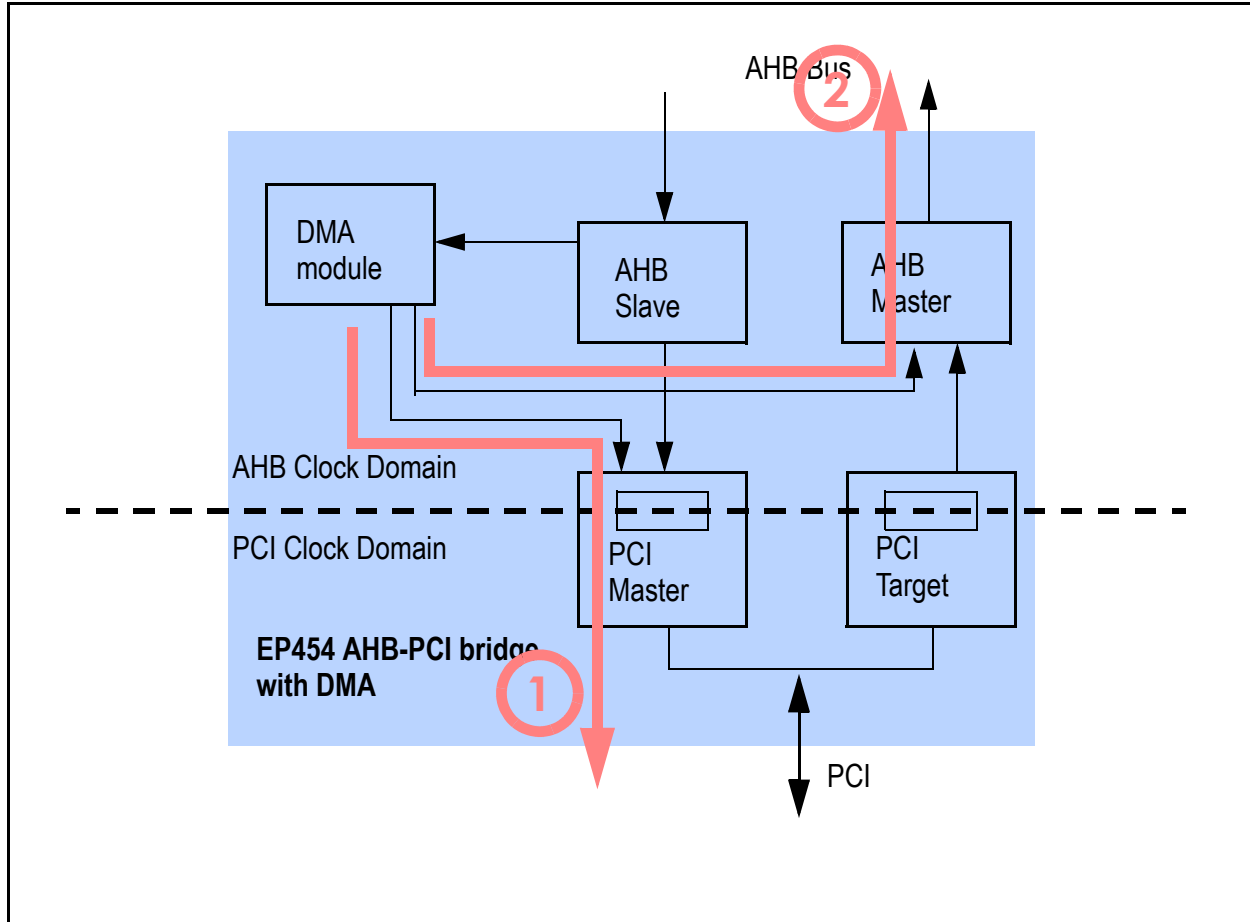


Step1: The DMA controller contains control registers. These registers should be programmed by the CPU in the AHB bus. The AHB slave receives the control register access from the AHB bus.

Step2. The control register access is then forwarded to the DMA controller by the AHB slave.



B.2.2 PCI to AHB DMA Transfer



After the DMA control registers is programmed, the DMA controller starts data transfer following these steps for PCI to AHB transfer:

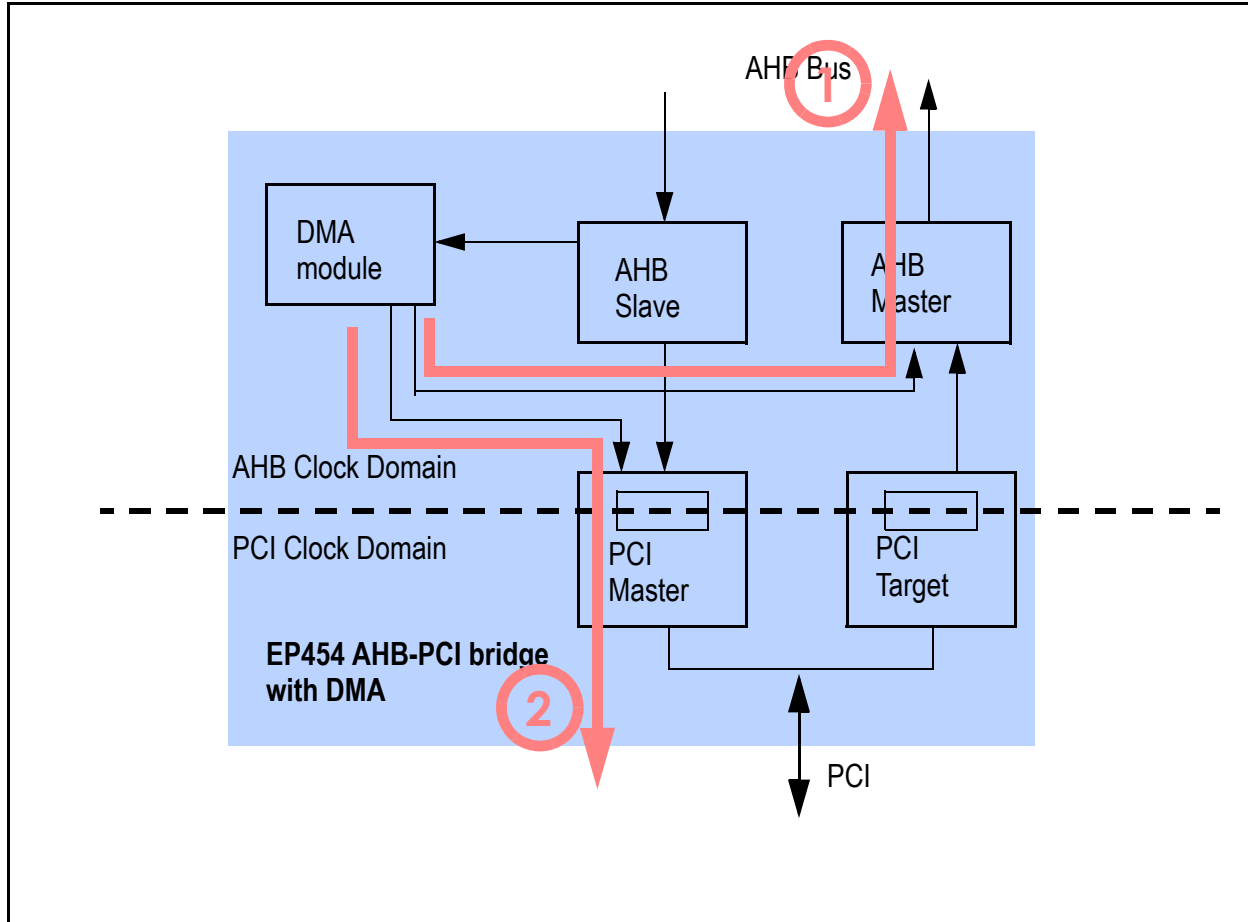
Step1: The DMA reads from the PCI bus. The request from the DMA is processed by the PCI master in EP454. The return data is stored in the DMA controller's data buffer.

Step2. The DMA then writes the data from the data buffer into AHB bus. The DMA sends the write request to the AHB master in EP454. The AHB master then writes the data to the AHB bus.

If the DMA transfer size is longer than 16 words, the DMA controller breaks the long transfer into multiple transfers of 16 words each to complete the transfer. The limit of 16 word burst transfer is an AHB bus limitation.



B.2.3 AHB to PCI DMA Transfer



After the DMA control registers is programmed, the DMA controller starts data transfer following these steps for AHB to PCI transfer:

Step1: The DMA reads from the AHB bus. The request from the DMA is processed by the AHB bus master in EP454. The return data is stored in the DMA controller's data buffer.

Step2: The DMA then writes the data from the data buffer into PCI bus. The DMA sends the write request to the PCI master in EP454. The PCI master then writes the data to the PCI bus.

If the DMA transfer size is longer than 16 words, the DMA controller breaks the long transfer into multiple transfers of 16 words each to complete the transfer. The limit of 16 word burst transfer is an AHB bus limitation.

B.3 DMA Control Registers

Each DMA channel contains of five control registers. The interrupt register which is common to all channels, is located after the control registers for the last chan-



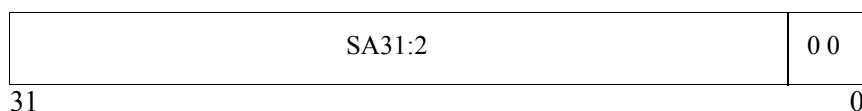
nel. The DMA control registers can be access either from the PCI bus, from the AHB bus or from a dedicated port. Please contact Eureka Technology for detail.

Table B.1: Control Register Mapping

Channel	Control Register
Channel 0	Source address
	Destination address
	Channel control register
	Transfer count
	Next DMA pointer
	Reserved
Channel 1	Source address
	Destination address
	Channel control register
	Transfer count
	Next DMA pointer
	Reserved
Common	Interrupt

B.3.1 Source Address

This register contains the source address. Address is incremented by 4 for each DMA word transferred.



B.3.2 Destination Address

This register contains the destination address. Address is incremented by 4 for



each DMA word transferred.

DA31:2	0 0
31	0

B.3.3 DMA Channel Control Register

Table B.2: Channel Control Register

Name	Bits	Description
	31	Reserved. Hardwired to zero
	30	Reserved. Hardwired to zero
	29	Reserved. Hardwired to one
	28	Reserved. Hardwired to zero
	27	Reserved. Hardwired to zero
	26	Reserved. Hardwired to one
TC	25	When set to a one, the DMA terminates data transfers when transfer count reaches zero and clears the STRT bit. When this bit is set to zero, the DMA transfer never finishes. The transfer count register wraps around to the maximum value when it reaches zero and continues the DMA transfer. When this bit is zero, DMA can be stopped by clearing the STRT bit, de-asserting DRQ#, or when the DMA encounters error.
INT	24	If the INT bit is set and the “Next DMA pointer” register is all zero (no more DMA chaining), the interrupt flag for the corresponding channel is set when the DMA finishes.
TYPE	23:21	Type of transfer: 0 1 0 : PCI to AHB transfer 0 1 1 : AHB to PCI transfer Others : Reserved
	20:19	Reserved. Hardwired to zero
CHG	18	Set CHG to one to enable modifying the STRT bit. This bit is automatically cleared after each write.

**Table B.2: Channel Control Register**

Name	Bits	Description
STRT	17	Set STRT to one to arm the DMA channel. The STRT bit can be modified only when CHG is set at the same time.
ERR	16	This bit is set to one by the DMA controller when it detects an error at the PCI interface or AHB bus. Once error is detected, the DMA terminates, ERR bit is set, STRT bit is cleared, and interrupt is generated. When a DMA is terminated due to error, the “Next DMA pointer” will not be read and no DMA chaining will be performed. This bit can be cleared by writing a “1” to this bit. Writing zero to this bit has no effect.
MAX-BURST	15:13	Maximum burst size: 000 : single 001 : burst of 2 010 : burst of 4 011 : burst of 8 100 : burst of 16
	12:2	Reserved. Hardwired to zero.
WORD-SIZE	1:0	Size of each DMA word. It is hardwired to “10” to indicate 32-bit data width

B.3.4 DMA Transfer Count

Table B.3: Transfer Count Register

Name	Bits	Description
	31:16	Reserved.
Transfer Count	15:0	This register contains the DMA transfer count. It is decremented by one for each DMA word transferred. The maximum transfer count is 64kwords.



B.3.5 Next DMA pointer

This register contains the next DMA pointer.

PTR31:2		0 0
31		0

The next DMA pointer is used by the DMA to read from memory to update the DMA control registers. 5 words are read from memory to load the control registers. The PTR value is incremented by 4 for each register it load. This register allows the software to setup a long chain of DMA transfers with virtually no limit on the chain length. The DMA stops the chaining process only when the next DMA pointer is all “0”.

B.3.6 Interrupt Status Register

Table B.4: Interrupt Status Register

Name	Bits	Description
	31:n	Reserved
INT_FLAG	n-1:0	One bit in each field represents one DMA channel. This bit is set by the DMA when interrupt is generated by the DMA controller. Interrupt may be generated due to normal completion of a DMA if INT bit is set or due to error. The INTR# output of the channel is asserted as long as this bit is one. This bit can be cleared only by writing into this register. Writing a “1” to any of these bits clears the bit. Writing zero has no effect.



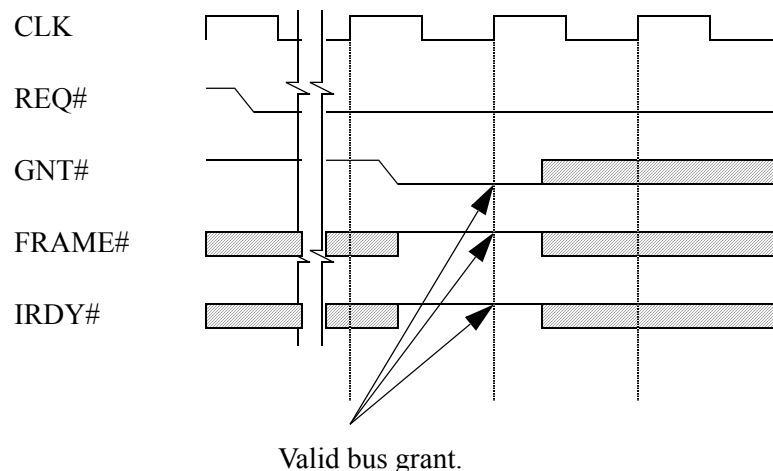
Appendix C. PCI Bus Arbiter

This appendix describes the PCI bus arbiter that can be added to the EP454 AHB-PCI bridge.

C.1 Bus Arbitration

When a bus master needs to execute a bus transaction, it asserts the bus request signal, REQ#, if the bus is not yet granted. REQ# is asserted by the master until the bus is granted. The bus is granted when GNT# to the master is asserted. The arbiter implements bus parking, which means that if none of the bus masters request the bus, a bus grant will still be issued to one of the bus master. The bus master which received the parked bus grant can start using the bus without asserting bus request.

The arbiter may de-assert a bus grant signal on any clock. A master is allowed to start a transaction at the rising clock edge after it samples bus grant active and the bus is in idle state. Bus is in idle state when both FRAME# and IRDY# are inactive.



C.2 Rotating Priority

The PCI bus arbiter is designed to arbitrate up to eight bus requestors. It has a choice of either fixed priority or rotating priority scheme. Fixed priority is non-



mally used only for special situation where higher priority must be reserved for certain devices on the system. It is usually provided on customer request.

Rotating priority arbitrates all the requestors on a fair and equal basis. With rotating priority, at any given time, the priority of each requestor is based on its position relative to the most recent bus user. Immediately after system reset, the priority is the same as if requestor 0 is the most recent master on the bus. If no request is active after reset, the bus is parked at requestor 0. The following table illustrate the priority scheme:

Most recent master on the bus	Request Port Number								
	Highest priority	2nd priority	3rd priority	4th priority	5th priority	6th priority	7th priority	Lowest priority	Bus parking
0	1	2	3	4	5	6	7	0	0
1	2	3	4	5	6	7	0	1	1
2	3	4	5	6	7	0	1	2	2
3	4	5	6	7	0	1	2	3	3
4	5	6	7	0	1	2	3	4	4
5	6	7	0	1	2	3	4	5	5
6	7	0	1	2	3	4	5	6	6
7	0	1	2	3	4	5	6	7	7

C.3 Bus Parking

The arbiter implements bus parking by designating one of the requestor as the default bus owner when no request is received. The default bus owner is the most recent bus master and it receives bus grant even though there is no bus request. The default bus owner can use the bus without asserted bus request as long as the bus grant is valid.

This bus parking method has the advantage that if there is only one bus master in the system, the bus is always parked to the only bus master.

Another advantage of this bus parking scheme is that once a transaction has started, if there is no other master requesting the bus, the current master would continue to receives bus grant for the duration of the transaction. This allows the current master to continue on a long data transfer without getting time-out by its latency timer.



C.4 Changing Bus Grant

Once bus grant is asserted to a requestor, it may be de-asserted by the arbiter and grant the bus to another requestor with higher priority.

If bus grant is de-asserted at the same clock FRAME# is asserted, the bus transaction is valid and will continue.

During bus idle state, when the arbiter de-asserted bus grant, it must wait one clock cycle before asserting bus grant to another bus requestor.

The arbiter recognize that a bus grant coincide with FRAME# or IRDY# asserted is not a valid bus grant.

C.5 Bus Time-out

The arbiter assumes that the current master is “broken” if it has not started an access after its bus grant signal has been asserted with bus request and the bus is in the idle state for 16 clock cycles. The arbiter ignores any broken master’s request and removes it from the arbitration process.

Once a bus master is considered broken, it must de-assert its bus request signals for one clock cycle before it is considered normal by the arbiter.

A bus master should start the transaction as soon as it receives bus grant from the arbiter. If it waits, the arbiter may de-assert its bus grant and grant the bus to another requestor. A master can start a bus transaction only if it receives bus grant at idle state in the immediate preceding clock cycle.



Appendix D. MiniPCI and Power management option

This appendix describes the optional Power Management & Mini-PCI support for PCI cores. The Mini-PCI and Power Management are separate options but typically are used together. The MiniPCI option is application when the EP454 is used as AHB-PCI host bridge or as a peripheral PCI master/target. The Power Management option is normally used only when the EP454 is a peripheral master/target.

Power Management support is compliant to PCI Power Management Interface Specification Rev 1.1. It is an optional features that can be included in Eureka Technology's standard PCI and Mini-PCI core.

The Power Management & Mini-PCI options are implemented by modifications internal to the PCI core, user interface, and its configuration registers. All modifications are include in the core before delivery to customer.

D.1 Configuration Register Modifications

PCI configuration register is included in the PCI core. To support power management, the configuration register block is expanded to include additional registers and fields of data. The following table describes the configuration registers with power management. New register locations are highlighted in yellow.

Table D.1: PCI Configuration Registers

Device ID		Vendor ID		00h
Status (Bit 4)		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1				14h
Base Address Registers 2				18h
Base Address Registers 3				1Ch
Base Address Registers 4				20h
Base Address Registers 5				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved			Cap_ptr	34h
Reserved				38h
Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch



Table D.1: PCI Configuration Registers

Power Management Capability (PMC)		Next Ptr	Cap. ID	location determined by Cap_ptr Cap_ptr + 4
Data	PMCSR-BSE Bridge Support extension	Power Management Control/Status Register (PMCSR)		

The following table describes the modified fields in detail.

Table D.2: Power Management Modified Fields

Offset	Bit	Name	Access	Description
6	4	Capability	R	This bit is hardwired to “1” to indicate the Power Management capability. Other bits of this register remains unchanged from the standard core.
34(hex)	7:0	Cap_ptr	R	This field provides the starting address of the capability link list. This field is provided by the user as PME_CAP input in the back-end bus. The PCI core uses this field to determine the location of the link list. The last 3 bits of the PME_CAP register must be zero so that the capability link list is properly aligned.
cap_ptr	7:0	Capability ID	R	This field is hardwired to 01h to indicate power management link list
cap_ptr + 1	7:0	Next Item Ptr	R	This field is hardwired to 00h to indicate last of the link list
cap_ptr + 2	15:0	Power Management Capability (PMC)	R	This field provides the operation characteristic of the PCI device. It contains information such as power saving level, current consumption, etc. This field is provided by the user logic as PME_PMC input to the back-end bus.



Table D.2: Power Management Modified Fields

Offset	Bit	Name	Access	Description
cap_ptr + 4	15	Power Management Command/Status Register	R/W/C	PME Status. This bit is set by the user logic when it intends to assert the PME# signal on the PCI bus. The user asserts the -PME_SET_STATUS input pin on the back-end bus to set this bit. This bit is cleared when a “1” is written by configuration write cycle. For system that does not assert PME# during D3cold state, this bit is reset during PCI reset. For system that supports PME# assertion during D3cold state, this bit is not affected by PCI reset. The core uses bit15 of PME_PMC input to determine if this support is needed. The PME_PMC bit 15 must be valid during PCI reset.
	14:13		R	Data Scale. This field is provided by the user on the back-end bus as the PME_DSC input.
	12:9		R/W	Data Select. This field can be read and write by configuration access, the PME_DSEL output pins on the back-end bus reflects the value of this register.



Table D.2: Power Management Modified Fields

Offset	Bit	Name	Access	Description
cap_ptr + 4	8	Power Management Command/Status Register	R/W	PME_EN. This field can be read and write by configuration access, the PME_EN output pins on the back-end bus reflects the value of this register. The PCI PME# signal is asserted when both this bit and the PME_status bit (bit 15) are asserted. For system that does not assert PME# during D3cold state, this bit is reset during PCI reset. For system that supports PME# assertion during D3cold state, this bit is not affected by PCI reset. The core uses bit15 of PME_PMC input to determine if this support is needed. The PME_PMC bit 15 must be valid during PCI reset.
	7:2		R	Reserved, hardwired to zero
	1:0		R/W	PowerState: This field can be read and write by configuration access, the PME_PSOUT output pins on the back-end bus reflects the value of this register written by configuration write. During configuration read, the PME_PSIN input from the back-end bus is returned to the PCI bus.
cap_ptr + 6	7:0	PMCSR-BSE bridge support extension	R	This field is hardwired to “0” since it is not a PCI-to-PCI bridge.
cap_ptr + 7	7:0	Data	R	Read only field provided by the back-end bus as PME_DT input.

D.2 Signal Modifications

The following table shows the signal modifications for power management



option. The “#” suffix in each signal name indicates active low signal. Please note that all signals in the following table operates in PCI clock domain.

Table D.3: Signal Modification

Name	Type	Side	Descriptions
CLKRUN#	I/O	PCI	The CLKRUN# signal is used by the system and the PCI card to communicate system clock requirement. CLKRUN# is an open drain output signal from the core.
MPCIACT#	I/O	PCI	Mini-PCI active. This is signal is asserted by the PCI device to indicate that its current function requires full system performance. MPCIACT# is an open drain signal when driven by the core.
PME#	O	PCI	This is an output signal on the PCI bus. It is asserted by the PCI core whenever the PME_EN and PME_status register bits are set to indicate power management event. PME# is an open drain output signal.
H_CRUNI#	I	Back-end	Input signal driven by the user logic to assert the CLKRUN# signal. The core asserts CLKRUN# when H_CRUNI# is asserted.
H_CRUNO#	O	Back-end	Output signal from the core to the user logic. It is a combinational signal reflecting the state of the CLKRUN# signal on the PCI.
H_MPCIACTI#	I	Back-end	Input signal from the back-end requesting assertion of the MPCIACTI# signal. This signal should be asserted by the user logic when it requires full system performance.
H_MPCIACTO#	O	Back-end	Output signal from the core to reflect the value of the MPCIACT# PCI signal.
PME_CAP[7:3]	I	Back-end	This signal is provided by the user to specify the capability pointer value. Bit 2:0 of the pointer is not provided by the user. These three bits are forced to “0” by the core.
PME_DSC[1:0]	I	Back-end	This signal provides the Data Scale input.



Table D.3: Signal Modification

Name	Type	Side	Descriptions
PME_DSEL[3:0]	O	Back-end	This output signal reflects the value of the Data Select field in the register.
PME_DT[7:0]	I	Back-end	This signal provides the Data value for configuration read.
PME_EN	O	Back-end	This signal reflects the value of the PME_EN bit in the configuration register.
PME_PMC[15:0]	I	Back-end	This signals provides the PMC values for configuration read.
PME_PSIN[1:0]	I	Back-end	This signal provides the PowerState value for configuration read.
PME_PSOUT[1:0]	O	Back-end	This signal reflects the PowerState value written by configuration write cycle.
PME_SET_STATUS	I	Back-end	This signal is asserted by the user to set the PME Status bit in the configuration register. The PCI core asserts the PME# output when the PME status and PME_EN bits are both set.

D.3 Functional Modifications

Power management support mainly involved in two areas: additional PCI signals and PCI configuration registers. The additional PCI signals are CLKRUN# and PME#. These two signals are controlled by the PCI core based on signals provided by the back-end bus.

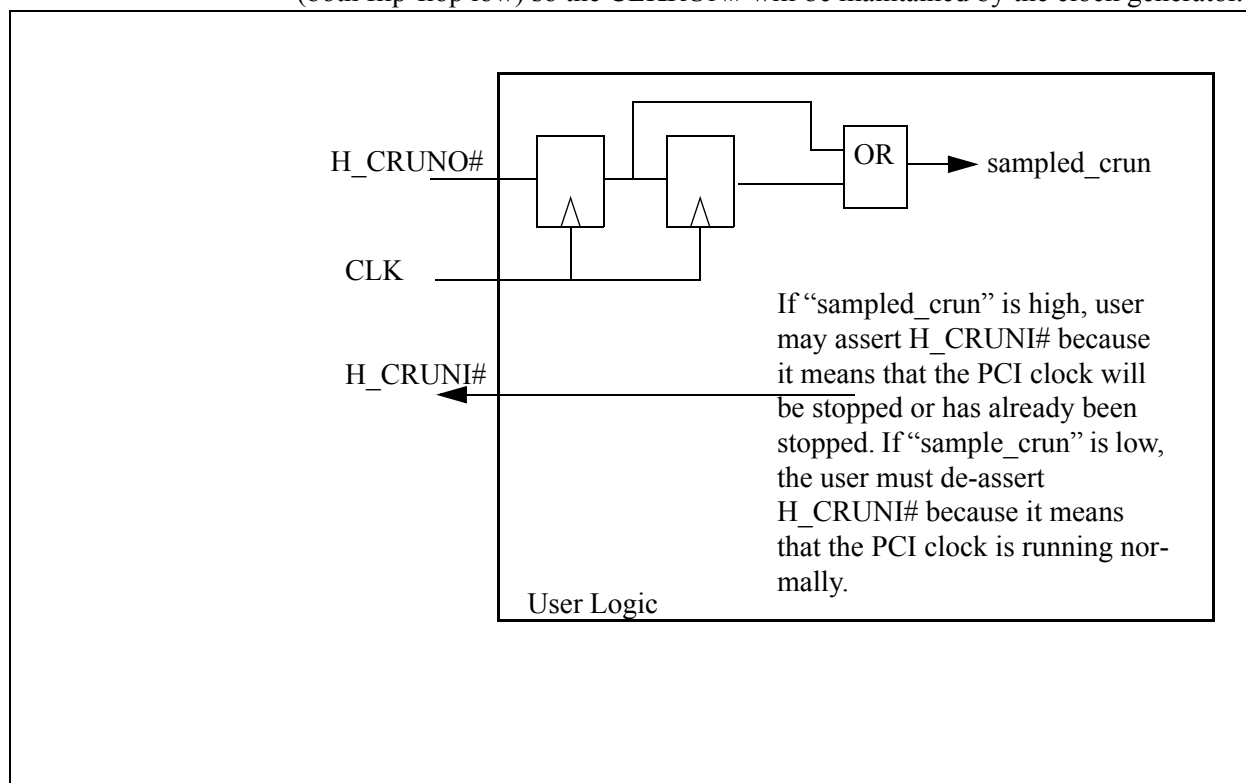
CLKRUN#

CLKRUN# is a bi-directional signal on the PCI bus. CLKRUN# is asserted by the system clock driver when the bus clock is running normally. It is de-asserted four cycles before the system clock is stopped or slowed down to inform all the PCI devices of the clock speed changes. Output from the core is controlled by the H_CRUNI# signal from the back-end. The user asserts the H_CRUNI# signals whenever it wants to inform the system to maintain the bus clock at the normal rate. The PCI core assert CLKRUN# on the PCI bus when H_CRUNI# is asserted. The H_CRUNO# output on the back-end bus reflects the value of the CLKRUN# signal and it can be used by the user logic.

The user logic is required to monitor H_CRUNO# before it asserts H_CRUNI#. It



should sample H_CRUNO# by two flip-flops in series inside the user logic code. If the output of either flip-flop is high, it indicates that the system is about to stop the clock or the clock has already been stopped. If the user wants to restart the clock, it should assert H_CRUNI# to request the system clock generator to start the clock. The PCI core asserts CLKRUN# when H_CRUNI# is asserted. Once the system clock is resumed, the flip-flop that samples H_CRUNO# in the user logic will sample H_CRUNO# and generate a “0” at its output. The user logic should de-assert H_CRUNI# after H_CRUNO# is asserted for two clock cycles (both flip-flop low) so the CLKRUN# will be maintained by the clock generator.

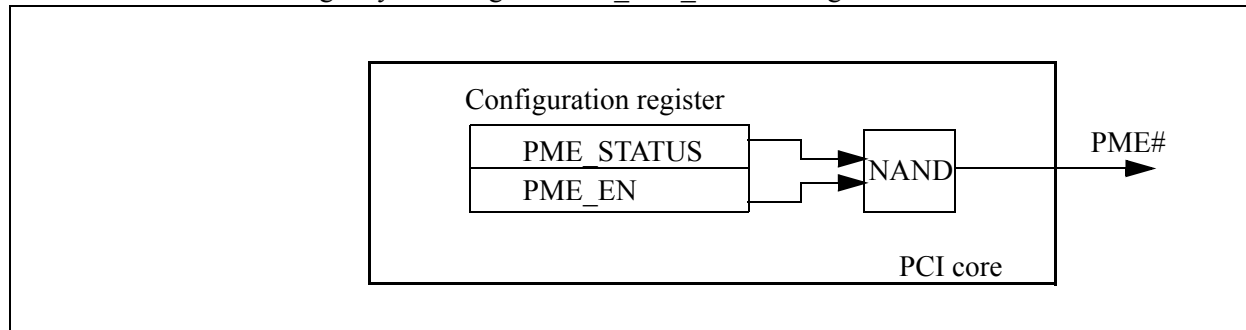


PME#

PME# is an open drain output from the core to the PCI bus. This signal is asserted by the core to request a change in its power management state or to report a power management event. It is asserted by the core when the PME Enable bit and the PME Status bit in the configuration register are both set. The PME Enable bit is set by software through configuration write. The PME Status bit is set by the user



logic by asserting the PME_SET_STATUS signal.



MPCIACT#

MPCIACT# is an open drain output from the core to the PCI bus. This signal is asserted by the core to indicate that its current function requires full system performance or requires that the system not enter any suspended state. It is asserted when the back-end H_MPCIACT# signal is asserted. Internal operation of the core is not affected by the MPCIACT# signal.

Configuration Register

Power management support requires additional configuration registers to be implemented by the core. The additional configuration registers are described in previous section of this document. These registers can be divided into three types: (1) read only register with read value hardwired inside the PCI core, (2) read only registers with read value provided by the back-end bus, and (3) registers which can be written by configuration write or by the back-end bus.

Type 1 register are handled by the core internally. They include the Capability Identifier, Next Item Pointer, PMCSR-BSE and all the reserved fields.

Type 2 registers require input value from the user side. The information required are very standard such as location of the link list or the power consumption level of the device. They include the Capability Pointer, PMC, PowerState(read), Data Scale and the Data field.

Type 3 registers can be written by software during configuration write to inform the back-end of certain system events, or by the back-end to assert the PME# signal. If they are writable by software, the software write value are output to the back-end. They include Data Select, PME_EN, and PowerState(write). The PME status register is the only the PME register that can be set by the back-end. It is set by the PME_SET_STATUS input to assert PME#.

D3cold state support

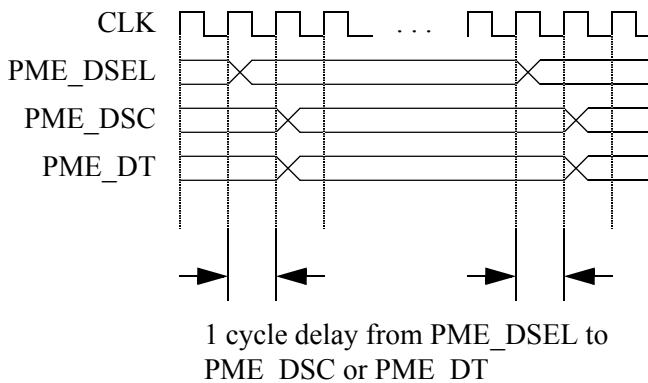
D3cold is one of the power levels allowed by PCI power management. If the core can assert PME# during D3cold state, it must retain the PME_STATUS and PME_EN register bit values during D3cold state. The user must continue to apply power to the core during D3cold state in order to maintain the logic level of these



register bits. Furthermore, when PME# is supported during D3cold state, the PME_STATUS and PME_EN register bits are not changed by PCI reset. If PME# during D3cold state is not supported, the core resets the PME_STATUS and PME_EN registers during PCI reset. The core uses bit 15 of PME_PMC input to determine the whether to reset these bits. The PME_PMC signals must be valid during PCI reset.

PME_DSEL, PME_DSC, PME_DT timing

PME_DSC and PME_DT are input data from the user logic. They are generated according the PME_DSEL output signal from the core. The following diagram shows the timing relationship between PME_DSEL to PME_DSC and PME_DT





Appendix E. 32-bit only Cardbus option for peripheral device

This appendix describes optional CardBus support for PCI master and target cores. This option is application only when the EP454 is used as a peripheral device instead of a host bridge core. When the EP454 is used as a host bridge core, please refer to the 32-bit only or 32/16-bit Cardbus option for host bridge. They are described in other sections of this document.

CardBus support is an optional features that can be included in standard Eureka Technology's PCI cores. CardBus architecture is modeled after the PCI bus architecture. With this CardBus support option in place, the PCI cores from Eureka Technology can be used in CardBus system environments as a CardBus PC Card master or target. All the features of the CardBus system, with the exception of LOCK, are supported by this option.

This option is implemented by modifications internal to the PCI core and its configuration registers, and by addition of CardBus specific signals to the core on the CardBus side and back-end interface.

E.1 CardBus Signal Modifications

CardBus and PCI bus shares many similar signals, even though they have different names. The following table shows the signal names when the CardBus option is implemented and their counterparts in the PCI bus environment.

Table E.1: CardBus Bus Interface Signals

Name		Type	Descriptions
CardBus	PCI		
CAD[31:0]	AD[31:0]	I/O	Address and Data are multiplexed on the same pins. A bus transaction consists of an address phase followed by one or more data phases. The core supports both read and write bursts.
CC/BE[3:0]#	C/BE[3:0]#	I/O	Bus Command and Byte Enables. They are multiplexed on the same pins. During the address phase of a transaction, C/BE# define the bus command. During the data phase, C/BE# define byte enables.
CCLK	CLK	I	System clock



Table E.1: CardBus Bus Interface Signals

Name		Type	Descriptions
CardBus	PCI		
CCLKRUN#	--	I/O	This is a CardBus only signal. It is implemented with the CardBus option. The CCLKRUN# signal is used by portable CardBus devices to request the system to turn on the bus clock.
CDEVSEL#	DEVSEL#	I/O	Device Select. When actively driven, indicates the driving device has decoded its address as the target of the current access. As an input, DEVSEL# indicates whether any device on the bus has been selected.
CFRAME#	FRAME#	I/O	Cycle Frame. It is driven by the master/target to indicate the beginning and duration of an access. FRAME# is asserted to indicate a bus transaction is beginning. While FRAME# is asserted, data transfers continue. When FRAME# is de-asserted, the transaction is in the final data phase.
CGNT#	GNT#	I	Grant. It indicates to the master/target that access to the CardBus/PCI bus has been granted.
---	IDSEL	I	Initialization Device Select. It is used as a chip select during configuration read and write transactions. This signal is connected externally to one of the AD signals according to PCI device number. This signal is used only in PCI core and does not exist in CardBus core.
CINT#	INTA#	O	CardBus/PCI interrupt signal. Controlled by back-end bus interrupt request.
CIRDY#	IRDY#	I/O	Initiator Ready. It indicates the master/target's ability to complete the current data phase of the transaction. IRDY# is used in conjunction with TRDY# to indicate a completion of one data transfer.

**Table E.1: CardBus Bus Interface Signals**

Name		Type	Descriptions
CardBus	PCI		
CPAR	PAR	I/O	Parity. It is an even parity across AD[31:0] and C/BE[3:0]# signals. It is driven by the master/target for address and write data phases, and is driven by the target for all read data phases.
CPERR#	PERR#	I/O	Parity Error. It is driven by the device on the PCI bus that receives data to report an error condition.
CREQ#	REQ#	O	CardBus/PCI Bus Request. Driven by the master/target to indicate to the arbiter that it needs to use the bus.
CRST#	RESET#	I	System Reset.
CSERR#	SERR#	O	System Error. It is driven by the device on the CardBus/PCI bus that receives address parity errors, data parity errors on the special cycle command, or any other system error.
CSTOP#	STOP#	I/O	Stop indicates the current target is requesting the master/target to stop the current transaction due to unusual conditions.
CTRDY#	TRDY#	I/O	Target Ready. Indicates the target's ability to complete the current data phase of the transaction. It is used in conjunction with IRDY#.

In addition to signals listed in table 1, there are other CardBus signals which are not generated by the core. These signals are either static signals or signals which should be generated directly by the user logic on the back-end bus.

Table E.2: CardBus Static/Direct Interface Signals

Signal	Type	Descriptions
CCD[2:1]# CVS[2:1]#	Static	These 4 signals are static signals and they should be hard-wired on the PC board. They are not connected to the core nor the chip where the core resides.

**Table E.2: CardBus Static/Direct Interface Signals**

Signal	Type	Descriptions
CAUDIO	O	This is an audio signal that can be used to carry a single amplitude digital waveform or pulse width modulated audio information. This signal, if used, should be generated by the user logic and connected directly to the CardBus connector.
CSTSCHG#	O	This is an optional signal that can be asserted by the CardBus device to signal status change. This signal, if used at all, should be generated by the user logic and connected directly to the CardBus connector.

E.2 Back-end Interface Modification

The optional CardBus supports affects the back-end interface by adding two signals to the back-end. Other back-end interface signals of the PCI core remain unchanged.

Table E.3: Additional Back-end Signals

Signal	Type	Descriptions
H_CRUNI#	I	Input signal driven by the user logic to assert the CCLKRUN# CardBus signal. The core samples H_CRUNI# at the rising clock edge and drives CCLKRUN# in the following clock cycle.
H_CRUNO#	O	Output signal from the core to the user logic. It is a combinational signal reflecting the state of the CCLKRUN# signal on the CardBus.

E.3 Configuration Register Modification

With the CardBus option, the PCI core supports a common set of the configuration registers between CardBus and PCI bus. All the PCI bus configuration registers are implemented together with the CardBus specific CIS pointer. The



following is a map of the configuration register space.

Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1 (as requested)				14h
Base Address Registers 2 (as requested)				18h
Base Address Registers 3 (as requested)				1Ch
Base Address Registers 4 (as requested)				20h
Base Address Registers 5 (as requested)				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved				34h
Reserved				38h
Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch

The CIS pointer is a read only register and has the following format:

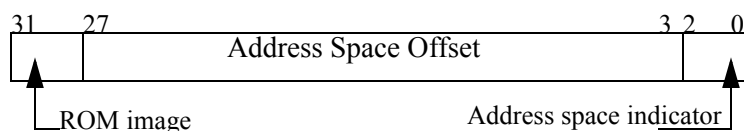


Table E.4: CIS Pointer Register Bit Assignment

Bits	Definition
31:28	Rom Image number. Defines the ROM image number in which the CIS is located. The offset is added to the start of the ROM image to identify the start of the CIS.



Table E.4: CIS Pointer Register Bit Assignment

Bits	Definition
27:3	Address Space Offset. Bit [2:0] define which space the CIS resides within. The address space offset is interpreted as follows for each type of address space: Configuration space - the offset is from location 40h in device specific region up to a maximum value of F8h. Memory space - the offset is from the base address specified by the selected base address register. Expansion ROM space - the offset value is from the start of the ROM image identified by bits[31:28].
2:0	Address Space Indicator. Specifies the base address within the space indicated. The offset (defined by bits[27:3]) is added to this base address to identify the start of the CIS. The address indicators values are: 0 = CIS is device-specific configuration space. 1 = CIS is in the memory pointed to by base address register 0 2 = CIS is in the memory pointed to by base address register 1 3 = CIS is in the memory pointed to by base address register 2 4 = CIS is in the memory pointed to by base address register 3 5 = CIS is in the memory pointed to by base address register 4 6 = CIS is in the memory pointed to by base address register 5 7 = CIS is in the device's expansion ROM. Bits[31:28] identify which ROM image.



Appendix F. 32/16-bit Cardbus Option for Host Bridge Device.

This appendix describes optional 16-bit and 32-bit CardBus support for PCI host bridge cores. This is an optional features that can be included in EP454 when this core is to be used as a host-to-cardbus bridge. This is option should not be used when the EP454 is used as a peripheral device.

PC Card architecture supports both 32-bit Cardbus mode and 16-bit PC Card mode. 32-bit CardBus is very similar to PCI specification so the 32-bit CardBus design is based on the PCI host bridge design.

16-bit mode operation (16-bit PC Card) is evolved from the PCMCIA specification. Both the 32-bit and 16-bits cards share the same physical connect specification but uses different signal assignments.

With this option, the PCI host bridge is be converted to a CardBus Host Bridge to support both 32-bit CardBus PC Card and 16-bit PC Card.

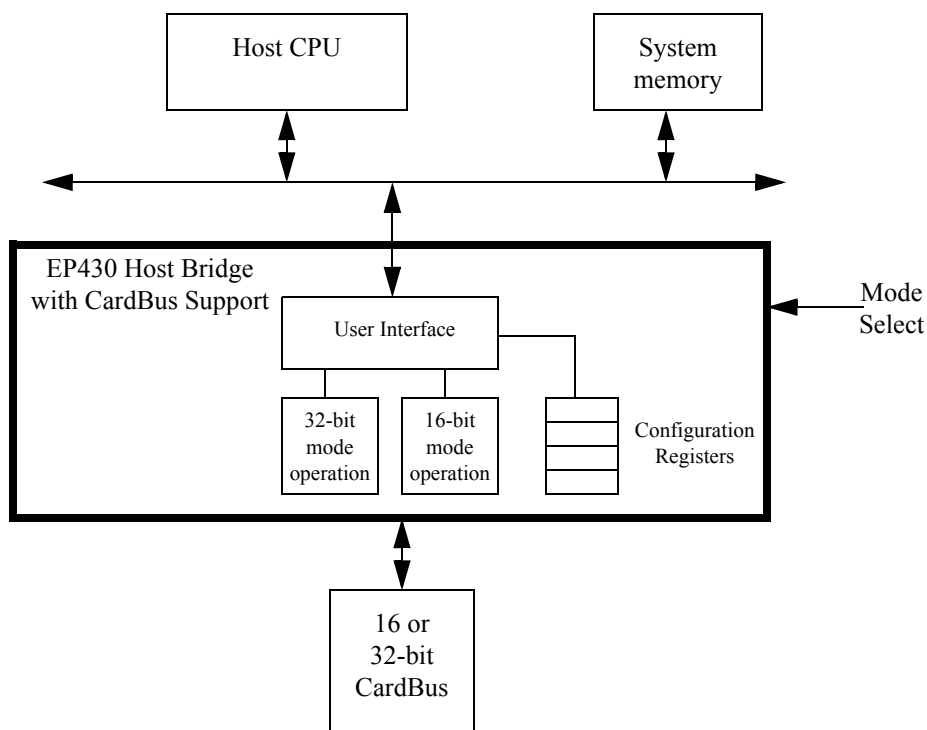
This application note describes only the effects of the CardBus option on the PCI core. For detail information regarding the operation of the standard PCI cores, please refer to the data sheet of the PCI cores.

Other options of the host bridge such as asynchronous clock domain can operate in conjunction with this CardBus option. But in order to highlight the effect due to the CardBus option, this document and the associated timing diagram assumes a single clock domain PCI host bridge.



F.1 Block Diagram

The following is a block diagram of this core in typical application.



F.2 Signal Modifications

Both 16-bit and 32-bit operation shares the same interface. Each card interface pin has different signal assignments according to operation mode. The following table lists the card interface signal sharing between the two operations modes.

Table F.1: Pin sharing in 16-bit PC Card and 32-bit CardBus

16-bit PC Card (memory)	16-bit PC Card (I/O)	32-bit CardBus	16-bit PC Card	16-bit PC Card	32-bit CardBus
D3	D3	CAD0	D1	D1	CAD29
D4	D4	CAD1	D2	D2	Reserved
D5	D5	CAD3	WP	IOIS16#	CCLKRUN#
D6	D6	CAD5	D11	D11	CAD2
D7	D7	CAD7	D12	D12	CAD4
CE1#	CE1#	CCBE0#	D13	D13	CAD6

**Table F.1: Pin sharing in 16-bit PC Card and 32-bit CardBus**

16-bit PC Card (memory)	16-bit PC Card (I/O)	32-bit CardBus	16-bit PC Card	16-bit PC Card	32-bit CardBus
A10	A10	CAD9	D14	D14	Reserved
OE#	OE#	CAD11	D15	D15	CAD8
A11	A11	CAD12	CE2#	CE2#	CAD10
A9	A9	CAD14	Reserved	IORD#	CAD13
A8	A8	CCBE1#	Reserved	IOWR#	CAD15
A13	A13	CPAR	A17	A17	CAD16
A14	A14	CPERR#	A18	A18	Reserved
WE#	WE#	CGNT#	A19	A19	CBLOCK#
READY	IREQ#	CINT#	A20	A20	CSTOP#
A16	A16	CCLK	A21	A21	CDEVSEL#
A15	A15	CIRDY#	A22	A22	CTRDY#
A12	A12	CCBE2#	A23	A23	CFRAME#
A7	A7	CAD18	A24	A24	CAD17
A6	A6	CAD20	A25	A25	CAD19
A5	A5	CAD21	RESET	RESET	CRST#
A4	A4	CAD22	WAIT#	WAIT#	CSERR#
A3	A3	CAD23	Reserved	INPACK#	CREQ#
A2	A2	CAD24	REG#	REG#	CCBE3#
A1	A1	CAD25	D8	D8	CAD28
A0	A0	CAD26	D9	D9	CAD30
D0	D0	CAD27	D10	D10	CAD31

32-bit CardBus and PCI bus shares many similar signals, even though they have different names. The following table compares the 32-bit CardBus signal with



regular PCI signals.

Table F.2: 32-bit CardBus Bus Interface Signals

Name		Type	Descriptions
CardBus	PCI		
CAD[31:0]	AD[31:0]	I/O	Address and Data are multiplexed on the same pins. A bus transaction consists of an address phase followed by one or more data phases. The core supports both read and write bursts.
CC/BE[3:0]#	C/BE[3:0]#	I/O	Bus Command and Byte Enables. They are multiplexed on the same pins. During the address phase of a transaction, C/BE# define the bus command. During the data phase, C/BE# define byte enables.
CCLKRUN#	--	I/O	This is a CardBus only signal. It is implemented with the CardBus option. The CCLKRUN# signal is used by portable CardBus devices to request the system to turn on the bus clock.
CDEVSEL#	DEVSEL#	I/O	Device Select. When actively driven, indicates the driving device has decoded its address as the target of the current access. As an input, DEVSEL# indicates whether any device on the bus has been selected.
CFRAME#	FRAME#	I/O	Cycle Frame. It is driven by the master/target to indicate the beginning and duration of an access. FRAME# is asserted to indicate a bus transaction is beginning. While FRAME# is asserted, data transfers continue. When FRAME# is de-asserted, the transaction is in the final data phase.
CGNT#	GNT#	O	Grant. It indicates to the master/target that access to the CardBus/PCI bus has been granted.



Table F.2: 32-bit CardBus Bus Interface Signals

Name		Type	Descriptions
CardBus	PCI		
---	IDSEL	I	Initialization Device Select. It is used as a chip select during configuration read and write transactions. This signal is connected externally to one of the AD signals according to PCI device number. This signal is used only in PCI core and does not exist in CardBus core.
CIRDY#	IRDY#	I/O	Initiator Ready. It indicates the master's ability to complete the current data phase of the transaction. IRDY# is used in conjunction with TRDY# to indicate a completion of one data transfer.
CPAR	PAR	I/O	Parity. It is an even parity across AD[31:0] and C/BE[3:0]# signals. It is driven by the master for address and write data phases, and is driven by the target for all read data phases.
CPERR#	PERR#	I/O	Parity Error. It is driven by the device on the PCI bus that receives data to report an error condition.
CREQ#	REQ#	I	CardBus/PCI Bus Request. Driven by the card to indicate to the arbiter that it needs to use the bus.
CSERR#	SERR#	O	System Error. It is driven by the device on the CardBus/PCI bus that receives address parity errors, data parity errors on the special cycle command, or any other system error. It is expected that the interrupt controller of the system monitors this signal directly in order to handle system error.
CSTOP#	STOP#	I/O	Stop indicates the current target is requesting the master to stop the current transaction due to Retry, Disconnect or Abort.

**Table F.2: 32-bit CardBus Bus Interface Signals**

Name		Type	Descriptions
CardBus	PCI		
CTRDY#	TRDY#	I/O	Target Ready. Indicates the target's ability to complete the current data phase of the transaction. It is used in conjunction with IRDY#.

In addition to signals listed in table above, there are other 32-bit CardBus signals which are not generated by the core. These signals are either static signals or signals which should be generated directly by the user logic on the back-end bus.

Table F.3: CardBus Static/Direct Interface Signals

Signal	Type	Descriptions
CCD[2:1]# CVS[2:1]#	Static	These 4 signals are static signals and they should be hard-wired on the PC board. They are not connected to the core nor the chip where the core resides.
CRST#/ RESET	O	This is the reset signal output to the card. This signal should be asserted by the system as part of the power-on sequence after the card is plugged in and the card type is detected.
CAUDIO	O	This is an audio signal that can be used to carry a single amplitude digital waveform or pulse width modulated audio information. This signal, if used, should be generated by the user logic and connected directly to the CardBus connector.
CBLOCK#	O	This core does not support the lock feature so this signal is not used. When operates in 32-bit Card bus mode, this signal is tri-stated.
CINT#	I	This signal is not sampled by the host bridge core. It should be connected directly to the interrupt controller of the system without going through the core.
CSTSCHG#	O	This is an optional signal that can be asserted by the CardBus device to signal status change. This signal, if used at all, should be generated by the user logic and connected directly to the CardBus connector.



The following table lists all the signals in 16-bit PC Card mode.

Table F.4: 16-bit PC Card Interface

Signal	Type	Description
ADDR[25:0]	O	Address lines
CE1#, CE2#	O	Card Enable These output signals are used both to select the card and to indicate to the card whether a byte or a word operation is being performed. CE2# always accesses the odd byte of the word. CE1# accesses the even byte or the Odd byte of the word depending on A0 and CE2#.
D15 - D00	I/O	These lines carry the data between the host and the card. D00 is the LSB of the Even Byte of the Word. D08 is the LSB of the Odd Byte of the Word.
INPACK#	I	This signal is not used in this mode.
IORD#	O	This is an I/O Read strobe generated by the host bridge. This signal gates I/O data onto the bus from the PC Card in I/O access.
IOWR#	O	The I/O Write strobe pulse is used to clock I/O data on the data bus into the card registers in I/O mode.
OE#	O	This is an Output Enable strobe generated by the host bridge. It is used to read data from the card in Memory Mode and to read the CIS and configuration registers.
READY (Memory Mode)	I	In Memory Mode this signal is set high when the card is ready to accept a new data transfer operation and held low when the card is busy. The Host memory card socket must provide a pull-up resistor. At power up and at Reset, the READR signal is held low (busy) until the card has completed its power up or reset function. No access of any type should be made to the card during this time.
IREQ# (I/O Mode)		I/O Operation – After the card has been configured for I/O operation, this signal is used as Interrupt Request. IREQ# is not processed by the core and should be routed directly to the system interrupt controller.



Table F.4: 16-bit PC Card Interface

Signal	Type	Description
REG#	O	Attribute Memory Select This signal is used during Memory Cycles to distinguish between Common Memory and Register (Attribute) Memory accesses. High for Common Memory, low for Attribute Memory. The signal must also be active (low) during I/O Cycles when the I/O address is on the Bus.
WAIT# (Memory Mode)	I	The WAIT# signal is driven low by the card to signal the host to delay completion of a memory or I/O cycle that is in progress.
WE#	O	This is a signal driven by the core to write data to the card in memory mode. It is also used for writing the configuration registers.
WP (Memory Mode) IOIS16# (I/O Mode)	I	This signal is not used by the core in memory mode. When the card or is configured for I/O Operation Pin 24 is used for the -I/O Selected is 16 Bit Port (IOIS16#) function. A Low signal indicates that a 16 bit or odd byte only operation can be performed at the addressed port.

The user interface remains mostly unchanged from the standard PCI host bridge core except for the following additions and modifications. Data transfer protocol is not affected by this option.

Table F.5: User Interface Signal Modifications

Signal	Type	Descriptions
H_32CARD	I	This signal selects between the 32-bit CardBus and 16-bit PC Card. If this signal is high, the card interface operates at 32-bit mode. If this signal low, the card interface operates as 16-bit mode.

**Table F.5: User Interface Signal Modifications**

Signal	Type	Descriptions
H_CRUNI#	I	Input signal driven by the user logic to assert the CCLKRUN# CardBus signal. The core samples H_CRUNI# at the rising clock edge and drives CCLKRUN# in the following clock cycle.
H_CRUNO#	O	Output signal from the core to the user logic. It is a combinational signal reflecting the state of the CCLKRUN# signal on the CardBus.

F.3 32-bit CardBus Clock

The clock input to the 32-bit CardBus shares the same signal bit as A16 in 16-bit PC Card. The host bridge core asserts drives the A16 signal in 16-bit PC Card mode. During 32-bit CardBus mode, this signal is tri-stated by the core. The core does not drive the clock signal on this pin because by doing that, it will introduce clock signal delay to the CardBus. The core operates assuming that the CardBus device receives the same clock signal (without skew) as it receives from the user interface. It is expected that the clock signal from the board level clock driver is routed to this pin during CardBus mode operations. During 16-bit PC Card operation, this clock driver should be tri-stated.

F.4 32-bit CardBus Operations

This section describes only the functional changes related to the CardBus PC Card addition. For the regular operation of the host bridge, please refer to the PCI host bridge data sheet.

When the bridge is configured as 32-bit CardBus (H_32CARD equals '1'), normal PCI master and target operations remain unchanged from the PCI bus.

F.4.1 Local Configuration Registers

The host bridge contains a complete set of PCI configuration registers and its operations are controlled by the configuration registers as defined in the PCI specification. These registers also provide information to the system regarding bus status such as parity error or bus abort.

CardBus interface does not contain the IDSEL input signal. If the external CardBus device initiates type 0 configuration read or write access, the host bridge will respond to these accesses as the target. However, the external CardBus device is not expected to initiate any configuration write access to modify the configuration



registers.

F.4.2 Host Bridge Initiates Configuration Access

The user interface can initiate configuration access on the CardBus through the host bridge. It uses the same mechanism as defined in the PCI specification. This mechanism is also called the “PC-AT compatible systems” and is also described in Eureka’s PCI host bridge data sheet.

Since the CardBus interface is a point-to-point connection and does not contain the IDSEL signal, one modification is necessary. When the user interface initiates a configuration access by accessing the CONFIG_ADDR and CONFIG_DATA registers, the host bridge checks the bus number and device number in the CONFIG_ADDR register. If bus number equals “0” and device number equals “0”, it assumes that its own configuration register is being accessed and the request will not be forwarded to the CardBus interface. The host bridge will process the configuration access locally.

If the bus number equals “0” but the device number is non-zero, the host bridge issues a type 0 configuration access on the CardBus interface. The external CardBus device will respond to the access according to the CardBus specification.

If the bus number is non-zero, the host bridge issues a type 1 configuration access on the CardBus interface. If the CardBus device is capable to accept a type 1 configuration, it will respond to the access.

With both type 0 or type 1 access, if the CardBus device does not respond to the access, the access will terminate as master abort and the host bridge will terminate the transfer on the user interface and update its configuration register according to PCI specification.

The host bridge does not respond to the configuration access on the CardBus if it is generated by itself.

F.5 16-bit PC Card Operations

When the bridge is configured as 16-bit PC Card (H_32CARD equals ‘0’), it functions as an initiator on the card interface. Access request from the user interface will be forwarded to the card interface using the 16-bit PC Card protocol.

The bus protocol on the user interface remains unchanged when 16-bit PC Card mode is selected. The user interface still follows the ADS#/RDY# protocol as in the PCI host bridge.

F.5.1 Space Select

16-bit PC Card has 3 address spaces: Memory, I/O and Attribute Memory. The following table maps H_CS# signal from the user interface to one of the following



3 spaces:

Table F.6: 16-bit PC Card Interface

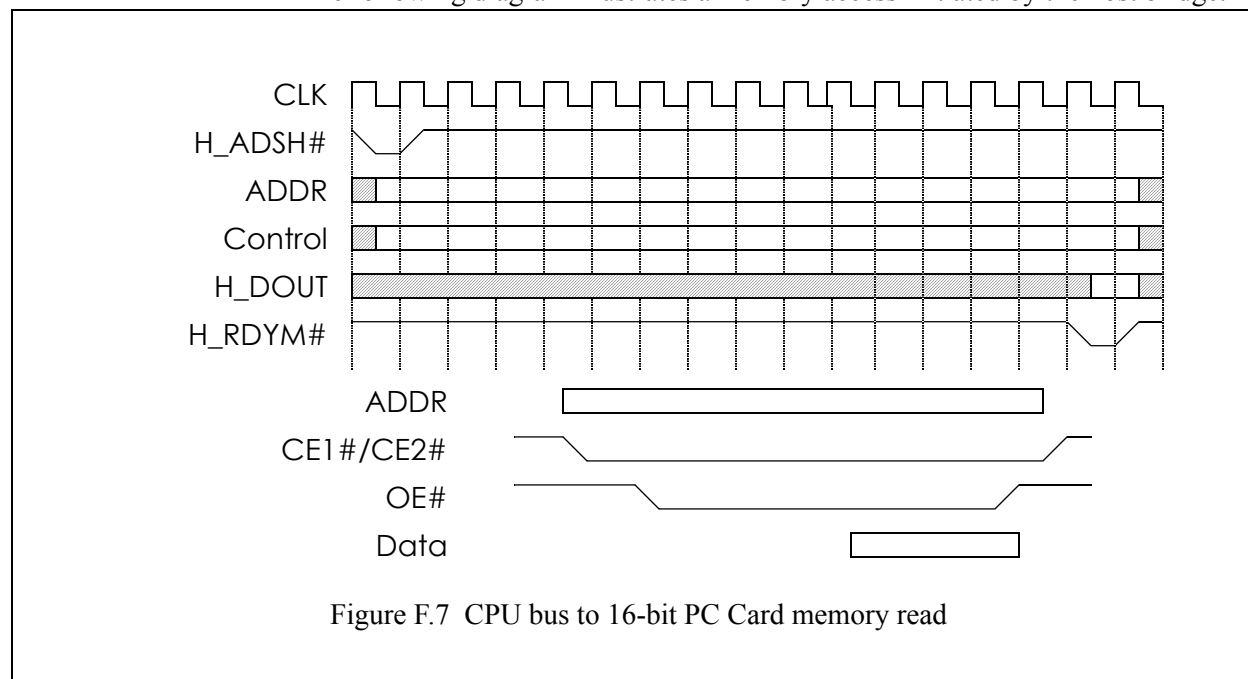
Signal	Type	Description
H_CS#[0]	I	PC Card Memory space
H_CS#[1]	I	PC Card I/O space
H_CS#[2]	I	PC Card Attribute Memory space

F.5.2 Burst Access

The host bridge supports burst access at the user interface. Any burst access or single access with data width wider than the 16-bit PC Card device is splitted into multiple access by the host bridge and issued to the PC Card. It is done automatically and without assistance from the user logic. This is to ensure that the user interface protocol remains unchanged with the PC Card operation.

F.5.3 Memory Access

The following diagram illustrates a memory access initiated by the host bridge.



The following diagram show a 16-bit PC Card memory write initiated by the



CPU.

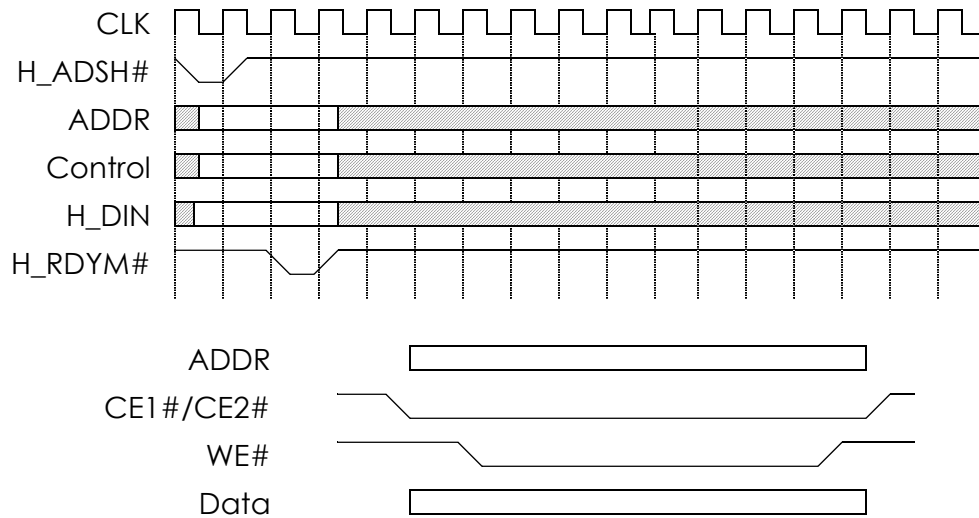


Figure F.8 CPU bus to 16-bit PC Card memory write



F.5.4 I/O Access

The following diagram show a 16-bit PC Card IO read initiated by the CPU.

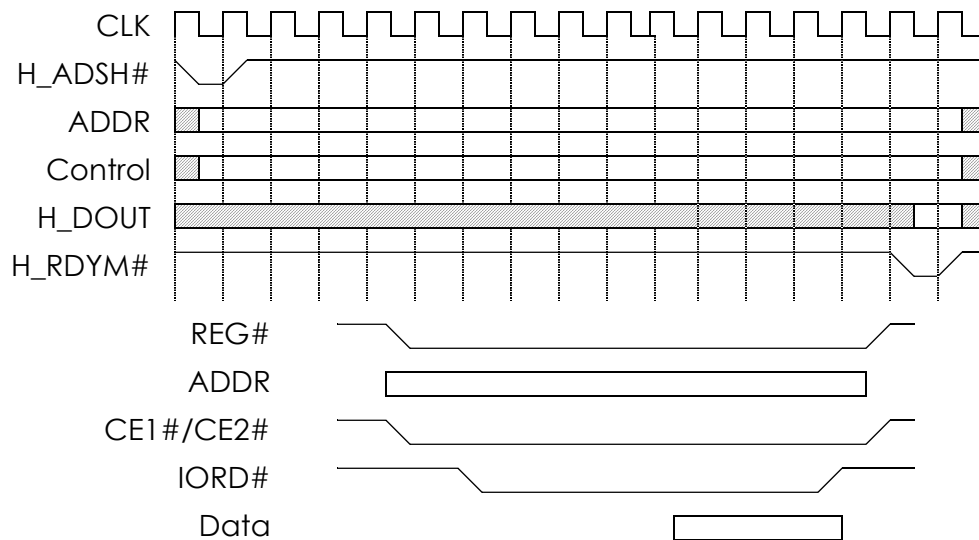


Figure F.9 CPU bus to 16-bit PC Card IO read



The following diagram show a 16-bit PC Card IO write initiated by the CPU.

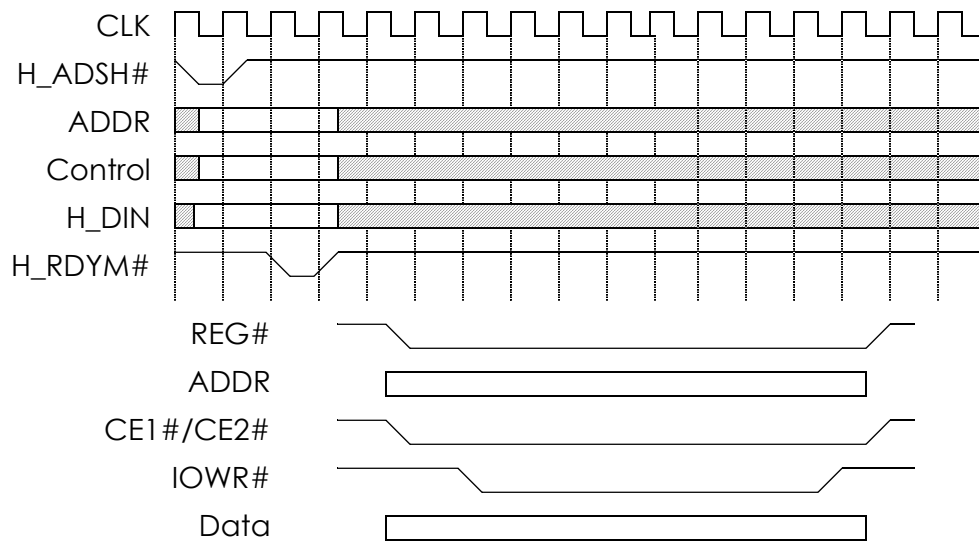


Figure F.10 CPU bus to 16-bit PC Card IO write



F.5.5 Attribute Memory Access

The following diagram show an attribute memory read initiated by the CPU.

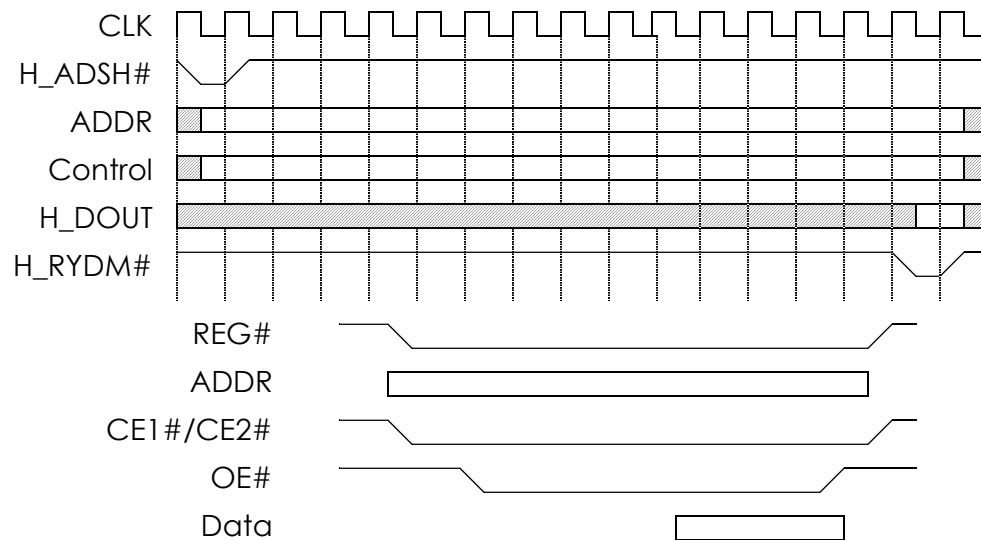


Figure F.11 CPU bus to 16-bit PC Card attribute memory read



The following diagram show an attribute memory write initiated by the CPU.

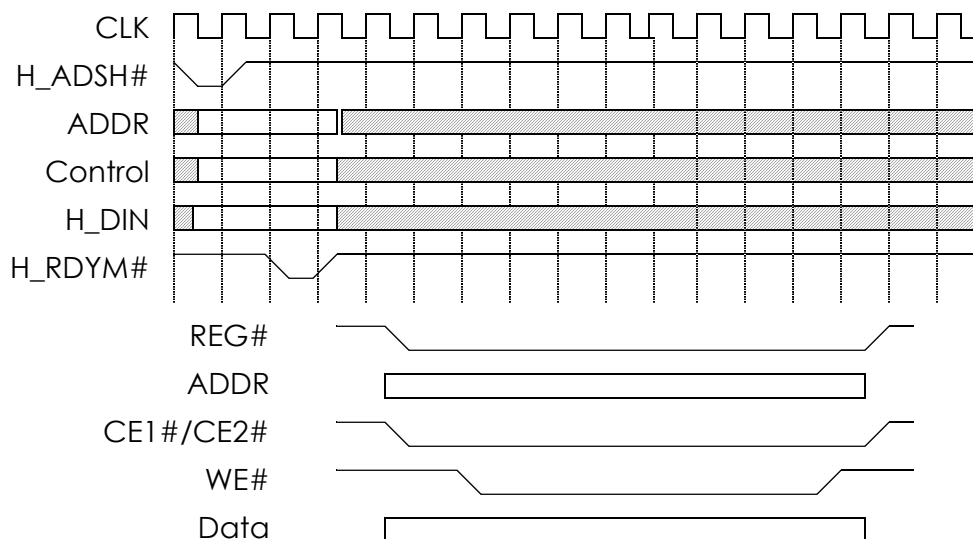


Figure F.12 CPU bus to 16-bit PC Card attribute memory write

F.6 Bus Arbiter

The standard PCI host bridge outputs PCI bus REQ# signal and receives GNT# signal from an external bus arbiter.

With the 32-bit CardBus option, the bus arbiter is included. The host bridge receives REQ# from the CardBus device when the CardBus device request bus mastership. The host bridge asserts the GNT# signal whenever it is not using the CardBus as a master.



Appendix G. Hot -Swap Option for peripheral device

This appendix describes the optional PCI standard Hot-Swap support for CompactPCI master and target cores. It is an optional features that can be included in Eureka Technology's standard PCI cores. For the EP454, this option is applicable only if the device is used as a peripheral master/target. This options is not applicable for a host bridge.

The Hot-Swap support option is implemented by modifications internal to the PCI core, back-end interface, and its configuration registers. All modifications are included in the core before delivery to customer.

G.1 Configuration Register Modifications

PCI configuration register is included in the PCI core. To support PCI Hot-Swap, the configuration register block is expanded to include additional registers and fields of data. The following table describes the configuration registers with power management. New register locations are highlighted in yellow.

Table G.1: PCI Configuration Registers

Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1				14h
Base Address Registers 2				18h
Base Address Registers 3				1Ch
Base Address Registers 4				20h
Base Address Registers 5				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved			cap_ptr	34h
Reserved				38h
Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch
	Hot Swap Control Register	Next Pointer	Capability Identification Register	Cap_ptr



The following table describes the modified fields in detail.

Table G.2: Hot Swap Modified Fields

Offset	Bit	Name	Access	Description
34(hex)	7:0	Cap_ptr	R	This field provides the starting address of the capability link list. This field is provided by the user as HS_CAP input in the back-end bus. The PCI core uses this field to determine the location of the capability list. The last 3 bits of the HS_CAP register must be zero so that the capability link list is properly aligned.
cap_ptr	7:0	Capability ID	R	This field is hardwired to 06h to indicate Hot-Swap capable
cap_ptr + 1	7:0	Next Item Ptr	R	This field is hardwired to 00h to indicate last of the link list



Table G.2: Hot Swap Modified Fields

Offset	Bit	Name	Access	Description
cap_ptr + 2	0	Reserved	R	Read only as '0'.
	1	ENUM_MASK	R/W	ENUM# mask. If this bit is "0", the core asserts ENUM# when REM_STAT or INS_STAT is one. This indicates an insertion or removal event. If this bit is "1", the core does not assert ENUM#
	2	Reserved	R	Read only as "0"
	3	LED	R/W	LED On/Off. This register bit can be read or written by software. The value of this register is reflected on the output signal LED_OUT.
	4:5	Reserved	R	Read only as "00"
	6	REM_STAT	R/C	Signal ENUM# removal event. The core sets this bit when L_STAT input is high and RESET_B is de-asserted, signaling an impending removal. This bit is cleared when a "1" is written by software. Writing zero has no effect on this bit. The value of this bit is reflected on the REM_STAT output from the core.
	7	INS_STAT	R/C	Signal ENUM# insertion event. The core sets this bit when L_STAT input is low and S_PRELD input is high. S_PRELD can be used to indicate completion of serial preload. This bit is cleared when a "1" is written by software. Writing zero has no effect on this bit. The value of this bit is reflected on the INS_STAT output from the core.



G.2 Signal Modifications

The following table shows the signal modifications for Hot-Swap option. The “#” suffix in each signal name indicates active low signal.

Table G.3: Signal Modification

Name	Type	Side	Descriptions
ENUM#	O	PCI	This signal indicates a Hot-Swap event. It is controlled by the ENUM_MASK, REM_STAT, and INS_STAT bits in the Hot-Swap configuration register.
HS_CAP[7:3]	I	Back-end	This signal is provided by the user to specify the capability pointer value. Bit 2:0 of the pointer is not provided by the user. These three bits are forced to “0” by the core.
INS_STAT	O	Back-end	Output signal reflecting the value of the INS_STAT bit in the Hot-Swap configuration register.
L_STAT_IN	I	Back-end	Hot-Swap local status input. This signal indicates the state of the ejector switch and is used by the core to control the INS_STAT and REM_STAT Hot-Swap configuration registers.
LED_OUT	O	Back-end	Output signal reflecting the value of the LED bit in the Hot-Swap configuration register.
REM_STAT	O	Back-end	Output signal reflecting the value of the REM_STAT bit in the Hot-Swap configuration register.
S_PRELD	I	Back-end	This signal indicates the completion of serial pre-load completion, indicating that the core is ready for host initialization. It is used by the core to control the INS_STAT bit in the Hot-Swap configuration register.

G.3 Functional Modifications

Hot-Swap support mainly involved PCI configuration register modification to support the additional registers that make up the capability list. Additional signals are provided in the PCI interface and the back-end interface to work with these



registers.

HS_CAP

HS_CAP is a static value provided by the back-end interface to indicate the location of the capability pointer. The capability pointer can be located anywhere outside the first 64 byte of the configuration space so bit[7:6] of the HS_CAP must not be “00”. Furthermore, to facilitate implementation, the capability list must be located at 8-byte boundaries. The 3 least significant bits are assumed to be “000”. The back-end interface only provides HS_CAP[7:3].

ENUM#

ENUM# is an open drain output from the core to the PCI bus. This signal is asserted by the core to indicate an insertion or removal event. This signal can be mask by the ENUM_MASK bit in the configuration register. The generation of this register is based on the REM_STAT and INS_STAT bits in the configuration register. Please see the configuration register section to see the rules governing the assertion of these bits.

Output signals

The value of the LED, REM_STAT and INS_STAT are driven by the core to the back-end interface. User logic can use these signals to control the LED or other functions as required.

Input signals

The value of the L_STAT_IN and S_PRELD signals are input to the core to control the Hot-Swap control registers. The L_STAT_IN is the input side of the L_STAT bi-directional signal. The S_PRELD signal indicates the serial preload completion of the device. Please see the configuration register section to see the usage of these bits by the core.



Eureka Technology Inc.
4962 El Camino Real
Los Altos, CA 94022, USA

Tel: 1 650 960 3800
Fax: 1 650 960 3805
email: info@eurekatech.com
<http://www.eurekatech.com>